

Increasing developers' code accountability perceptions in open source software development

Sebastian Clemens Bartsch^{a,*}, Jan-Hendrik Schmidt^a, Martin Adam^b, Alexander Benlian^a

^a Technical University of Darmstadt, Hochschulstr. 1, Darmstadt 64289, Germany

^b Georg-August-University of Göttingen, Platz der Göttinger Sieben 5, Göttingen 37073, Germany

ARTICLE INFO

Keywords:

Code accountability
Code quality
Developers
Mixed-method research approach
Open source software projects

ABSTRACT

Open source (OS) software projects rely on high-quality code, making developers' participation in reviewing their code essential to improve its quality. In non-OS domains, formal consequences (i.e., sanctions and rewards like fines or bonuses) are usually applied to raise developers' perceived accountability for their code, thus motivating them to review it. However, in the OS domain, such formal consequences are often absent or only weakly pronounced, raising questions of whether and how developers perceive accountability for their code and how it affects their motivation to improve its quality. Using a developmental mixed-method research approach, we conducted a qualitative study with semi-structured online interviews to develop a research model ($n = 29$) and validated it through a follow-up scenario-based factorial survey ($n = 480$). Our findings indicate that in the OS domain, developers often perceive themselves accountable for their code. Project- and individual-related factors, such as legal liability for contributed code, the integration of the OS software project into other projects, the work status of developers, and gaining a reputation for code contributions, appear as mechanisms to influence and increase developers' perceived code accountability. Furthermore, we observe that perceived code accountability promotes developers' intention to review their code before contributing it to OS software projects and, thus, contribute code with higher quality. Overall, our paper contributes to information systems (IS) research on accountability perceptions in OS software development, indicating that in the OS domain, formal consequences such as sanctions and rewards are not exclusively necessary for evoking developers' perceived code accountability and increasing code quality.

1. Introduction

OS software projects are integral to the implementation of algorithmic systems (e.g., Recker & La Rosa, 2012; Rolandsson, Bergquist, & Ljungberg, 2011), with over 75 % of organizations relying on them for the development and deployment of their algorithmic systems (Schnaack & Termer, 2023). OS software offers significant benefits for organizations, including cost and time savings through free access to continuously evolving code (Fitzgerald, 2006; Goggins, Lombard, & Germonprez, 2021; Wolter, Barcomb, Riehle, & Harutyunyan, 2023), fostering innovation, interoperability, and reduced dependency on proprietary software. However, inconsistent code quality in OS software poses substantial operational and security risks for organizations (e.g., Gobert, Nagy, Rocha, Demeyer, & Cleve, 2023; Goggins et al., 2021; Synopsys Incorporation, 2023). For example, in 2022, an analysis of

over 1500 OS software projects found vulnerabilities in more than 85 % of them (Synopsys Incorporation, 2023), with critical examples including *Log4j*, *libssh*, and *Apache Struts 2*, all of which resulted in serious security breaches (Apache Software Foundation, 2017; Newman, 2021; Perez-Etchegoyen, 2023; Red Hat Incorporation, 2018).

Given these serious risks, strengthening review practices before code contributions is essential. Many vulnerabilities result from unintentional mistakes that could have been prevented through more rigorous scrutiny of the code, particularly by enhancing self-review practices before code contributions (e.g., Wieringa, 2020). Since developers are most familiar with their own code, reviewing their own code enables developers to detect and correct defects more efficiently than peer reviewers, ultimately leading to higher code quality (e.g., Wang et al., 2020; Yin, Yuan, Zhou, Pasupathy, & Bairavasundaram, 2011). However, developers often lack motivation for thorough code reviews (Ada & Majid, 2022), a

* Corresponding author.

E-mail addresses: bartsch@ise.tu-darmstadt.de (S.C. Bartsch), schmidt@ise.tu-darmstadt.de (J.-H. Schmidt), martin.adam@uni-goettingen.de (M. Adam), benlian@ise.tu-darmstadt.de (A. Benlian).

<https://doi.org/10.1016/j.ijinfomgt.2025.102974>

Received 29 August 2024; Received in revised form 30 August 2025; Accepted 30 August 2025

Available online 9 September 2025

0268-4012/© 2025 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

challenge that is particularly pronounced in OS software projects, where contributions are typically voluntary and neither monetarily rewarded nor managerially supervised (Dalle, David, Ghosh, & Wolak, 2004; Fitzgerald, 2006). Consequently, identifying mechanisms that encourage self-reviews is critical for enhancing code quality and mitigating security risks in OS software projects.

One promising approach to motivating developers to engage in self-review practices is to foster perceived code accountability, which refers to the obligation to explain and justify one's code to relevant stakeholders (e.g., Rouse, 2007; Krogh, Haefliger, Spaeth, & Wallin, 2012; Wieringa, 2020). In the OS domain, perceived code accountability implies that developers need to explain and justify the behavior and functionality of their code, particularly to users integrating the code into their algorithmic systems (e.g., Novelli, Taddeo, & Floridi, 2023; Wieringa, 2020). In proprietary software development contexts, accountability is enforced through formal consequences such as sanctions and rewards, including financial penalties, job loss, promotion, or performance-based bonuses (e.g., Bovens, 2007; Martin, 2019), which motivates developers to uphold code quality, including rigorous code reviews (e.g., Finnigan & Gross, 2007; Martin, 2019). However, in the OS domain, such formal enforcement mechanisms are often absent or only weakly pronounced. OS developers are often volunteers, lack monetary incentives, and are shielded from legal liability by software licenses (e.g., Apache License, 2004; Dalle et al., 2004; Fitzgerald, 2006; MIT, 1987). These circumstances within the OS domain raise the question of how OS developers can still be motivated to perceive themselves as accountable for their code and, consequently, improve their code quality. In this context, informal consequences (i.e., consequences that are not legally enforceable but affect developers on a personal level, such as reputation loss) may serve as alternative mechanisms to enhance perceived code accountability in OS software projects. By leveraging informal accountability pressures, it may be possible to compensate for absent or only weakly pronounced formal consequences, leading developers to a stronger commitment to high-quality code.

Although OS software projects with high code quality are indispensable for organizations, existing IS research has focused on investigating how developers can be motivated to contribute to OS software projects (e.g., Hertel, Niedner, & Herrmann, 2003; Sharma, Daniel, Chung, & Grover, 2022; Krogh et al., 2012; Wu, Gerlach, & Young, 2007) rather than investigating "how and why OS software developers produce high-quality software" (Krogh et al., 2012, p. 666). In this context, prior IS research has acknowledged the role of perceived accountability in improving code quality (e.g., Novelli et al., 2023; Wieringa, 2020) but knows little about how developers can be made aware of code accountability in the OS domain, where formal consequences are often absent or only weakly pronounced. This gap in prior IS research leads us to the following research questions:

RQ1: *How is developers' perceived code accountability shaped in the OS domain?*

RQ2: *How does developers' perceived code accountability affect code quality in OS software projects?*

To examine these research questions, we drew on accountability literature and employed a mixed-method research approach with a developmental purpose (Venkatesh, Brown, & Sullivan, 2024). We first conducted qualitative interviews with 29 OS developers to identify key factors influencing perceived code accountability. Based on these insights, we developed a research model, which we then validated through a scenario-based factorial survey with 480 participants. Our findings reveal that developers' perceived code accountability in OS software development arises from a complex interplay of project- and individual-related factors. Specifically, we identify four key factors that enhance developers' perceived code accountability: (1) legal liability awareness through software licenses, (2) contribution to OS software projects widely integrated into other projects, (3) contributions made as part of a professional relationship, and (4) gaining high reputation within the OS community. These factors significantly enhance

developers' perceived code accountability, which, in turn, strengthens their intention to review their code, thereby encouraging them to conduct activities that improve the quality of their code contributions.

Building on these findings, our research offers four important contributions to IS research on accountability in OS software development. First, while prior IS research has focused on formal consequences as a means to enhance accountability perceptions (e.g., Ahmed & van den Hoven, 2010; Martin, 2019), our findings show that project- and individual-related factors can also increase developers' perceived code accountability, especially in the OS domain where formal consequences are absent or only weakly pronounced as developers typically contribute voluntarily without any liability. By shifting the focus from formal enforcement mechanisms to a broader set of accountability drivers, our study challenges existing perspectives on accountability perceptions and provides a more comprehensive understanding of how accountability perceptions are shaped. Second, our research extends previous IS research, which has primarily examined motivational factors driving OS software contributions (e.g., Hornuf & Jeworrek, 2023; McIntosh, Kamei, Adams, & Hassan, 2016; Krogh et al., 2012) but has largely overlooked mechanisms for improving code quality. Our findings highlight perceived code accountability as a key factor influencing developers' intention to review their code, thereby enhancing code quality in OS software projects. This finding is particularly crucial as it underscores the need to further explore how accountability perceptions shape developer behavioral intentions and software quality outcomes. Third, and more broadly, our study underscores the critical role of perceived code accountability within OS governance. Integrating accountability perceptions into OS governance structures adds an additional governance mechanism to enhance code quality, complementing existing community-driven practices such as peer reviews. This contribution addresses a significant research gap by providing actionable strategies for enhancing OS software governance through accountability frameworks (Krogh et al., 2012). Fourth, by focusing on the OS software context, we obtain context-specific insights that build upon and extend prior findings from accountability theory (Vance, Lowry, & Eggett, 2013). These insights help refine accountability theory for decentralized, volunteer-driven environments, while also demonstrating its broader applicability across different organizational and technological settings.

2. Theoretical background and related literature

2.1. Commonalities and differences between open source and proprietary software development

To understand how perceived code accountability operates in different software development environments, it is essential to contrast OS with proprietary software development, as their structural differences shape how accountability and governance mechanisms work. OS software refers to software "that allows for the modification of source code, is freely distributed, is technologically neutral, and grants free subsidiary licensing rights" (Aksulu & Wade, 2010, p. 577). While this openness of source code fosters collaboration and innovation, it also introduces challenges in governance and quality control. Traditionally, OS software projects rely on voluntary code contributions from a community of developers (Dalle et al., 2004; Fitzgerald, 2006; Perens, 1999). In this voluntary and community-driven environment, the governance of OS software projects depends on informal mechanisms such as non-mandatory peer reviews and shared community norms to uphold code quality (e.g., Rigby, German, & Storey, 2008). Consequently, developers typically have no legal or contractual obligations to fix defects or maintain their code contributions over time, as OS software projects are not contractually bound to provide ongoing support.

However, this traditional, voluntary, and community-driven view of OS software development only captures part of contemporary OS software development. In practice, many prominent OS software projects

are not solely maintained by volunteers but are also supported by commercial organizations (e.g., [Bubb, 2025](#); [Fitzgerald, 2006](#); [Wolf, 2022](#)). For example, OS software projects such as Android and Kubernetes (backed by Google), React (backed by Meta), or Visual Studio Code (backed by Microsoft) illustrate that organization-backed contributions are present, but coexist with volunteer-driven efforts ([Google, 2008](#); [Google, 2014](#); [Meta, 2013](#); [Microsoft Corporation, 2015](#)). In such organization-backed OS software projects, core developers are frequently full-time employees operating under contractual obligations and professional accountability standards. This coexistence of community-driven and organization-backed contributions makes the OS software landscape more heterogeneous, while still firmly rooted in the principles of openness and collaborative development.

Thus, OS software development shares commonalities and has differences to other software development contexts, particularly proprietary software development, which refers to developing software that is developed and owned by individuals or organizations who maintain strict control over its distribution, usage, and modification ([Anand, Krishna, Tiwari, & Sharma, 2018](#)). Similarities rely on reaching the overarching goal of providing reliable, high-quality code. Both settings engage in practices like code review, testing, and quality assurance. The crucial distinction, however, lies in the degree of formalization and the locus of control. In proprietary software development, these practices are institutionalized through contractual obligations and legal requirements, enforced by the software owner who controls access to the code ([Anand et al., 2018](#); [Singh, Bansal, & Jha, 2015](#)). Therefore, these developers operate under formal accountability mechanisms, such as legal liability and financial incentives ([Bovens, 2007](#); [Martin, 2019](#)). In contrast, the OS software environment operates on a spectrum. While traditional community-based OS software projects rely on voluntary norms, the growing number of organization-backed OS software projects introduces more formalized structures to ensure high code quality.

Consequently, to gain a holistic and honest view of OS software development, it is insufficient to restrict the focus to the traditional voluntary and community-driven model of OS software development. Contemporary OS software development is also shaped by organizations, while still retaining unique features such as transparent code bases and permissionless contribution rights (e.g., [Aksulu & Wade, 2010](#); [Fitzgerald, 2006](#)). Thus, it is likely that factors associated with proprietary software development may now coexist in OS software development, albeit in a less pronounced and more heterogeneous form. [Table 1](#) summarizes the key commonalities and differences between OS and proprietary software development.

2.2. Open source software in algorithmic systems' implementation

Despite differences in governance and quality assurance compared to proprietary software, the widespread use of OS software in algorithmic systems highlights its strategic value to organizations. Organizations frequently leverage OS software to reduce development costs and avoid financial and time investments required for creating or purchasing proprietary alternatives (e.g., [Boehmke, Hazen, Boone, & Robinson, 2020](#); [Fitzgerald, 2006](#)). However, by integrating OS software into their algorithmic systems, organizations also assume the risks associated with its potential vulnerabilities, as OS software projects do not always adhere to the same stringent quality assurance standards as proprietary software (e.g., [Fitzgerald, 2006](#); [Pistilli, Muñoz Ferrandis, Jernite, & Mitchell, 2023](#); [Synopsis Incorporation, 2023](#)). Ensuring that OS software is developed with high code quality is therefore essential, not only for enhancing security and reliability but also for maintaining competitiveness with proprietary solutions (e.g., [Boulanger, 2005](#); [Krogh et al., 2012](#)).

However, in the OS domain, motivating developers to contribute code with high quality is challenging as the licensing frameworks under which OS software is distributed explicitly exclude liability, implying that developers are not legally liable for defects or harm resulting from

Table 1

Commonalities and Key Differences Between Proprietary and OS Software Development.

Dimension	Proprietary Software Development	OS Software Development
Code Access	Restricted and controlled by the software owner.	Different in OS software development, as the code is publicly accessible to anyone.
Contribution Rights	Limited to authorized employees or contractors.	Different in OS software development, as contributions are permissionless, so that anyone can contribute without prior approval.
Governance mechanisms	Formalized through contracts, policies, and standards.	Different in OS software development as the governance is community-driven, defining its own values and norms.
Legal Liability	Developers/ organizations are subject to liability for defects.	Usually absent, but may occur in organization-backed OS software projects.
Defect Resolution	Fixed by the owner.	Different in OS software development, as defects are fixed by the willingness of the community. Otherwise, self-fixing of defects is necessary.
Employment Status of Contributors	Mostly employed developers with contractual obligations.	Usually volunteers, but a mix of volunteers and employed developers in organization-backed OS software projects can exist.
Quality Assurance	Institutionalized (including, for example, mandatory audits, standardized testing, and compliance).	Usually informal and community-driven, with (non-mandatory) peer reviews, but institutionalized mandatory code reviews can exist in organization-backed OS software projects.

their code contributions (e.g., [Apache License, 2004](#); [MIT, 1987](#)). Consequently, developers lack incentives to mitigate defects and harm by providing high-quality code. As a result, organizations adopting OS software bear the full risks of potential defects, necessitating rigorous evaluation and testing of the OS software project before deployment. This evaluation and testing are crucial for identifying and mitigating defects before organizations integrate OS software projects into their algorithmic systems, where faulty code could have serious operational or ethical implications.

However, systematically verifying the quality of OS software poses significant challenges for organizations. Many OS software projects comprise thousands of lines of code, making the identification of defects and their subsequent correction highly complex and resource-intensive (e.g., [Allamanis & Sutton, 2013](#); [Mockus, Fielding, & Herbsleb, 2000](#)). As a result, when open-source software is already developed and includes high-quality contributions, organizations benefit by reducing the necessity for extensive post-adoption reviews, thereby minimizing associated costs. This underscores the importance of fostering mechanisms that encourage OS developers to prioritize code quality during development, ultimately enhancing the security and reliability of OS software for algorithmic systems.

2.3. Open source governance to increase code quality

OS governance plays a pivotal role in sustaining OS software projects by establishing mechanisms for directing, controlling, and coordinating code contributions ([Markus, 2007](#); [O'Mahony & Ferraro, 2007](#)). A central objective of OS governance is to ensure high code quality, thereby minimizing defects and enhancing OS software reliability (e.g., [Aberdour, 2007](#); [O'Mahony & Ferraro, 2007](#)). To foster high-quality code contributions, OS governance structures are typically designed to be open and meritocratic ([Capra, Francalanci, & Merlo, 2008](#); [Capra,](#)

Francalanci, Merlo, & Rossi-Lamastra, 2011; O'Mahony & Ferraro, 2007), where developers gain social status based on the quality of their contributions rather than the number of code commits (O'Mahony & Ferraro, 2007). This system incentivizes developers to prioritize quality over quantity, reinforcing a culture where high-quality contributions enhance developers' reputations within the OS community.

While OS governance provides structural incentives for high-quality contributions, the actual realization of high-quality code largely depends on developers' engagement in reviewing their own and others' code (e.g., Kononenko, Baysal, & Godfrey, 2016; McIntosh et al., 2016). Code review is essential for defect identification and correction, yet it is often not a primary focus for developers (Aberdour, 2007; Midha & Bhattacharjee, 2012). Since most developers contribute code voluntarily, there are no formal supervisory structures to enforce rigorous review practices. As a result, motivating developers to actively engage in code review remains a critical challenge for ensuring code quality (e.g., Rashid, Clarke, & O'Connor, 2019).

To address this challenge, many OS software projects incorporate peer review requirements into their OS governance structures, making code review a prerequisite for merging code contributions into the main codebase (e.g., Aberdour, 2007; Rigby et al., 2008). These OS governance mechanisms, functioning as quality control, ensure that contributed code undergoes scrutiny before being published. However, the effectiveness of peer reviews is contingent upon the engagement and diligence of participating developers (e.g., Aberdour, 2007; Rigby et al., 2008). Inconsistent or superficial peer review can allow defects to remain undetected, increasing the risk of deploying flawed code.

Given these limitations, complementary mechanisms are needed to further enhance code quality. Self-review practices, where developers systematically examine their own code before submission, represent a promising approach to improving code quality. By proactively identifying and correcting defects before the peer review stage, developers reduce the likelihood of flawed code being merged and published. Unlike peer review, self-review is fully within the developer's control, making it an independent and potentially more reliable mechanism for ensuring code quality. Despite its potential, existing IS research has paid little attention to how developers can be motivated to engage in self-review. However, prior studies suggest that holding developers accountable for their code contributions may serve as an effective incentive to enhance code quality (e.g., Rouse, 2007; Krogh et al., 2012). Understanding how perceived code accountability influences developers' motivation to review their own code within OS software projects could provide valuable insights into improving the code quality of OS software projects.

2.4. Developers' code accountability in open source software projects

Code accountability refers to the obligation of developers to explain and justify their code to relevant forums, such as managers and users (Novelli et al., 2023; Wieringa, 2020). In non-OS domains, formal consequences ensure accountability by equipping stakeholders with enforcement measures, such as financial sanctions or rewards, to incentivize developers to meet quality standards and provide explanations when required (Bovens, 2007). As a result, developers behave in intended ways and can be assumed to proactively review their code to prevent errors that might necessitate justification or to reactively examine their code contributions when explanations are demanded (e.g., Bovens, 2007; Novelli et al., 2023; Wieringa, 2020). However, in the OS domain, formal consequences are often absent or only weakly pronounced, as developers typically contribute their code voluntarily, are not bound by employment contracts, and are protected by software licenses that shield them from legal liability (e.g., Dalle et al., 2004; Fitzgerald, 2006; MIT, 1987). This distinction makes it challenging to apply insights from non-OS domains, where formal consequences drive accountability perceptions, to OS software projects.

Despite the absence or weakly pronounced formal consequences,

prior IS research has emphasized the importance of perceived code accountability in OS software projects (e.g., Ågerfalk & Fitzgerald, 2008; AlMarzouq, Grover, & Thatcher, 2015; Midha & Bhattacharjee, 2012), particularly for the organization of OS software projects and its collaboration with external stakeholders (e.g., Germonprez, Levy, Kendall, & Kendall, 2020; Midha & Bhattacharjee, 2012). However, much of this research assumes that developers inherently accept code accountability without investigating how accountability perceptions emerge when formal consequences are absent or only weakly pronounced. As a result, little is known about alternative mechanisms that could compensate for missing or weakly pronounced formal consequences in OS software projects.

This lack of understanding of alternative mechanisms may stem from IS research's predominant reliance on accountability theory as the primary theoretical lens for studying accountability perceptions (e.g., Adam, 2022; Bartsch, Milani, Adam, & Benlian, 2024). Accountability theory posits that accountability perceptions are shaped by the four factors, namely, identifiability (the degree to which an individual's contributions can be traced back to them), social presence (the sense of being observed or connected to others), expectation of evaluation (the belief that one's contributions will be assessed), and awareness of monitoring (the recognition that activities are tracked) (Vance et al., 2013). Previous IS research has typically applied this theory in contexts where formal consequences are present (e.g., Adam, 2022; Vance, Lowry, & Eggett, 2015). However, even in contexts where formal consequences are absent or only weakly pronounced, accountability theory may remain a valuable lens for understanding, for example, how developers perceive themselves as accountable for their code in the OS software context.

Applying accountability to OS software projects, identifiability might be triggered through public commit histories and attribution in public repositories, social presence might be fostered via community interactions on platforms such as GitHub, expectations of evaluation might emerge from issue discussions linked to one's contributions, and awareness of monitoring might be reinforced through automated testing and error tracing of one's code contributions. At the same time, the OS software context differs from other software development contexts, such as proprietary software development, in ways that introduce additional mechanisms influencing code accountability perceptions. Strengthening these perceptions could improve OS governance by motivating developers to take greater accountability for their code contributions, particularly in activities such as self-review, thereby directly improving code quality (e.g., McIntosh et al., 2016; Krogh et al., 2012).

Although perceived code accountability has been recognized as a motivational factor in algorithmic system development (e.g., Bartsch et al., 2024), IS research has largely overlooked the OS software context in terms of accountability, which differs from the previously investigated research contexts in which formal consequences are typically present. Consequently, insights into fostering accountability in environments without predominantly existent formal enforcement mechanisms remain limited (e.g., Horneber & Laumer, 2023; Martin, 2019; Schmidt, Bartsch, Adam, & Benlian, 2025). Furthermore, IS research has primarily focused on motivating developers to participate in OS software projects, emphasizing factors such as human capital development and personal fulfillment (e.g., Hertel et al., 2003; Sharma et al., 2022). However, less attention has been given to motivating developers to engage in quality-enhancing activities such as reviewing one's own code (e.g., Kononenko et al., 2016; McIntosh et al., 2016; Krogh et al., 2012). Understanding how perceived code accountability emerges in the OS domain is thus essential for evaluating its influence on developer behavior, particularly in improving code quality.

3. Research studies

3.1. Developers' perceived code accountability and mixed-method research approach

We examined code accountability in the OS domain through the lens of developers' perceived code accountability, allowing us to explore how developers recognize and internalize their accountability obligations. This approach enables an individual-level analysis by capturing developers' subjective perceptions of their code accountability. By focusing on perceived accountability, we make the concept tangible and measurable, providing empirical insights into how developers experience and respond to accountability perceptions within OS software projects.

To examine developers' perceived code accountability, we adopted a mixed-method research approach (Fink, 2022; Venkatesh et al., 2024) with a developmental purpose by first conducting online interviews followed by a scenario-based factorial survey. This sequential mixed-method research approach allowed us first to compensate for the limited insights of IS research into understanding developers' perceived code accountability in domains with usually absent or only weakly pronounced formal consequences by developing a research model using qualitative data. Additionally, since qualitative data lack statistical robustness (e.g., Möhlmann, Zalmanson, Henfridsson, & Gregory, 2021), we employed a scenario-based factorial survey to provide statistical robustness to the developed research model and test respective hypotheses. By following this developmental mixed-method research approach, we leveraged the advantages of both research methods and increased the validity and robustness of our overall results. Fig. 1 presents our mixed-method research approach to achieve our research objectives, which we adapted from previous IS research (Picoto, Bélanger, & Palma-dos-Reis, 2014).

3.2. Study 1: interview study to develop a research model

3.2.1. Interviews and participant demographics

We conducted 29 semi-structured online interviews with developers actively contributing to prominent OS software projects like *TensorFlow* (Abadi et al., 2015) and *Spack* (Gamblin et al., 2015). After our 29th interview, we stopped conducting more interviews as we obtained indications for data saturation by observing significant overlap in the statements of the interviewees and received a comprehensive and extensive data set (Keutel, Michalik, & Richter, 2014). To identify relevant projects, we utilized GitHub as a search platform for OS software projects (GitHub, 2007). OS software projects were then selected based on their popularity, as indicated by the number of stars they received. We subsequently contacted developers involved in these OS software projects, inviting them to participate in voluntary online interviews. Our interviews lasted, on average, 32:33 min (SD =

10:09 min), were recorded digitally, and transcribed for further analysis with the developers' verbal declarations of consent. The interviews explored developers' perceived code accountability in the OS domain, particularly focusing on how this perception arises and how this perception affects their code development activities. For example, as part of the interviews, we asked the participating developers questions like "Why do you perceive yourself accountable for your code when participating in OS software projects?" and "How does your perceived accountability for your code affect your behavior and activities when participating in OS software projects?". Table 8 in the appendix displays the complete semi-structured interview guide that we used for all our interviews.

Although these interview questions did not delve into personally sensitive areas, we exercised particular care in handling the resulting ethical concerns. In doing so, we provided full disclosure to the developers regarding the interview process and ensured their anonymity in the analysis. In addition, we emphasized that there were no right or wrong answers to prevent developers from steering them in a predefined direction and to reduce the social desirability bias. In this way, we provided developers with a safe and open space to tell us about their OS software development experiences without directing them into ethically questionable or unpleasant situations.

Finally, the socio-demographical analysis of our sample reveals that 82.76 % of the interviewees contribute code to OS software projects voluntarily in their free time and have an average of 10.28 years (SD = 4.80) of OS software development experience. Moreover, the interviewees have participated in, on average, 70.93 (SD = 91.89, median = 38) OS software projects, with an average number of 10.62k (SD = 18.97k, median = 3.98k) code commits. Finally, the socio-demographical analysis revealed that 96.55 % of the interviewees identified themselves as male, 3.45 % as female, and 0.00 % as non-binary. Thus, we identify an imbalance between male and female interviewees. Although this gender distribution represents the typical proportion of female OS developers of approximately 3.00 % (Filippova, Forte, Galvez, Weiss, & Dabbish, 2017), we took particular care in treating possible biases that result from gender imbalance in our subsequent analysis. Table 2 lists the socio-demographics of our interviewees, which we coded as OSD01 to OSD29.

3.2.2. Analysis

We employed an inductive qualitative content analysis method to analyze our data (Corbin & Strauss, 2014; Elo & Kyngäs, 2008). First, we conducted comprehensive reviews of each interview, keeping our research question in mind. This review involved a thorough consideration of the socio-demographics and backgrounds of the interviewees, particularly in relation to OS software development, to better understand and interpret their statements. Subsequently, we began the analysis by starting with open coding. During open coding, we established a set of evaluation categories designed to systematically classify and label

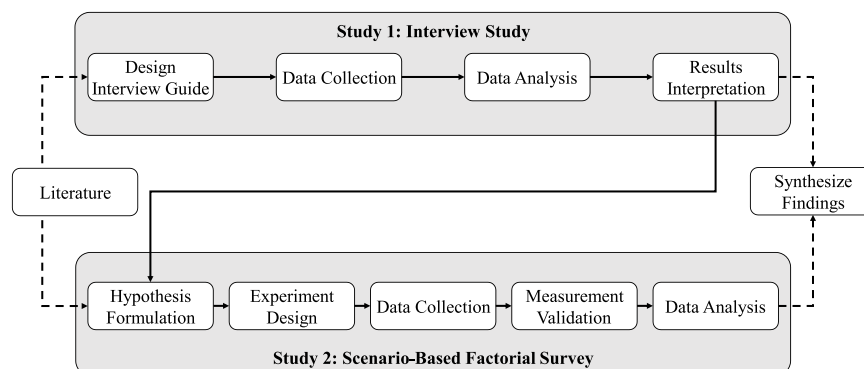


Fig. 1. Structure of the Mixed-Method Research Approach.

Table 2
Socio-Demographics of the Participants in Study 1.

Code	Gender	Work Status	OS Software Development Experience in Years	Number of Participated OS Software Projects	Number of Contributions to OS Software Projects	Interview Duration in Minutes
OSD01	m	v	10	5	3.07k	30
OSD02	m	p	17	471	31.62k	41
OSD03	m	v	6	81	3.77k	33
OSD04	m	v	13	73	12.84k	47
OSD05	m	v	5	110	2.23k	20
OSD06	m	v	12	54	4.39k	27
OSD07	m	v	6	19	1.36k	13
OSD08	m	v	11	130	8.52k	17
OSD09	m	v	11	95	16.20k	28
OSD10	m	p	6	31	5.95k	25
OSD11	m	v	6	35	1.40k	24
OSD12	m	v	16	34	5.06k	40
OSD13	m	v	15	117	6.00k	34
OSD14	m	v	8	48	1.23k	26
OSD15	f	v	6	12	2.65k	25
OSD16	m	v	8	28	1.50k	23
OSD17	m	v	19	22	1.39k	58
OSD18	m	p	8	38	1.76k	31
OSD19	m	v	7	11	1.42k	24
OSD20	m	v	18	55	0.99k	32
OSD21	m	v	8	86	11.72k	32
OSD22	m	v	4	28	1.19k	34
OSD23	m	v	5	8	3.98k	49
OSD24	m	p	7	10	1.73k	46
OSD25	m	v	12	65	17.19k	33
OSD26	m	v	10	66	25.41k	34
OSD27	m	v	24	268	95.23k	47
OSD28	m	p	10	27	2.75k	42
OSD29	m	v	10	30	1.38k	29

Note: m = male, f = female, v = contributing code voluntarily in their free time, p = contributing code as part of a professional relationship

the relevant statements from the interviews. These categories finally served as the first-order indicators for our data.

We then proceeded to the axial coding process (Corbin & Strauss, 2014). Within this process, we sorted and clustered the first-order indicators based on their relevance and relation to each other. This sorting and clustering process enabled us to identify distinct second-order categories, providing a more comprehensive perspective on the first-order indicators and facilitating a deeper understanding of the relationships between them. This process of sorting and identifying new categories was iterative, involving continuous refinement and additional labeling as new characteristics emerged from the data. Once we identified no further second-order categories, we derived themes from these categories and finally synthesized them into core categories. Any inconsistencies in our coding process were meticulously discussed among the authors' team until we achieved consensus. Table 3 illustrates the resulting data structure affecting developers' perceived code accountability.

3.3. Study 1: results

The interviews provided nuanced insights into developers' perceived code accountability and its influence on their activities during OS software development. In particular, we identified project- and individual-related factors shaping developers' perceived code accountability. Furthermore, developers' perceived code accountability is associated with behavioral intentions within OS software development, in which heightened code accountability perceptions increase their intention to review their code contributions and thus improve the quality of code contributions.

3.3.1. Perceived code accountability

3.3.1.1. Project-related factors. Legal Liability: Developers' perceived code accountability depends on their recognition and understanding of the legal liability that arises from the software license used within the OS

software project. Most common OS software licenses (e.g., MIT and Apache 2.0) explicitly disclaim legal liability for developers, stating that the code is provided "as is" with no guarantees or warranties (Apache License, 2004; MIT, 1987). These disclaimers are intended to protect contributing developers from any legal liabilities arising from the use of their contributed code. While such software licenses serve to encourage participation in OS software projects by reducing developers' legal risks (e.g., Colazo & Fang, 2009), the way developers perceive and interpret these software licenses in terms of their legal liability varies and affects their perceived code accountability.

Many developers lack sufficient knowledge about the implications of software licensing, often resulting in the assumption that the liability lies primarily with the user and not with themselves. This assumption leads developers to only deal with the legal implications to a limited extent without knowing exactly what is behind the software licenses. Against this backdrop, developers often emphasize that, in parallel to legal liability, code accountability lies with users when incorporating the OS software project into their algorithmic systems. Thus, developers allocate both the legal liabilities and the code accountability to the users, making them perceive less accountability for their code. However, while these assumptions regarding legal liability and code accountability are often true, developers told us that they would benefit from clearer communication of the implications of software licenses within OS software projects to better understand their role in assuming legal liability and code accountability. In this context, one developer noted:

"To be honest, only in certain cases, except if I have to use an open source software project as part of my work, then I have actually to care about the license, but in general, I do not excessively care about the licensing as long as it is open source." (OSD10)

Despite this, the recognition of software licenses varies: Only a minority of developers possess a thorough understanding of software licenses, recognizing the exclusion of any legal liabilities resulting from their contributed code. This recognition often results from a clear

Table 3

Data Structure of Project- and Individual-Related Factors Affecting Developers' Perceived Code Accountability.

First-Order Indicator	Second-Order Categories	Themes	Core Categories
• Citing legal mechanisms of software licenses that protect developers from any formal consequences • Finding that developers often do not care about the legal aspects of OS software projects • Talking about quitting the OS software project when the software license changes, and holding developers accountable for their code • Providing examples of software licenses within OS software projects	Legal Liability		
• Differentiate the implications between contributing code to OS software projects with few versus OS software projects with many integrations into other projects • Highlight the important role of developers in preventing potential harm occurring to users who utilize their OS software projects • Talking about expectations of users regarding their behavior during OS software projects and the contributed code • Talking about the objectives followed by the OS software projects	Project Integration	Project-Related Factors	
• Differentiate the implications between contributing code in free time versus contributing code as part of a professional relationship • Express that small OS software projects typically do not follow the objective of providing high code quality • Talking about missing compensation for coding efforts • Emphasizing that developers can do whatever they want when contributing code in their free time	Work Status		Perceived Code Accountability
• Emphasizing the importance of reputation outside the OS community • Explaining that a high reputation opens up new opportunities for developers in their professional careers • Discussing the linkage between contributing code with high quality and high reputation • Providing examples to gain a high reputation	Reputation	Individual-Related Factors	

Note: The listed first-order indicators serve as examples. Other first-order indicators are not included here for brevity but exhibit similar characteristics to those listed.

communication of the implications of the software licenses or the developers' own interest in dealing with software licenses. Developers who were affected by clear communication or have personally dealt with software licenses tend to perceive lower levels of accountability for their contributed code. They defeat their lower levels of perceived code accountability by rationalizing that the exclusion of any legal liabilities minimizes their need to explain or justify any potential downstream consequences resulting from their contributed code. In this context, one developer stated the following:

"Currently, if you're doing open source work, you're not going to get any kind of consequences. [...] Even people who have injected malicious code in popular packages, I think they got away with nothing since there isn't really a legal structure there to be had." (OSD29)

Conversely, developers with little familiarity with software licenses lack a nuanced understanding of potential legal liabilities arising from these software licenses. Developers often justify their limited engagement with software licenses by citing their primary focus on coding rather than legal regulations, and missing clear communication about the implications of the software licenses. As a result, we observe that developers who do not fully understand software licenses in terms of their legal liabilities often believe that they might be held accountable for explaining or justifying their code contributions in hypothetical future scenarios, fostering a heightened sense of perceived code accountability.

We therefore argue that if the legal liability of developers is clearly asserted and brought into developers' sphere of attention, for example, by clear communication, developers are less likely to perceive themselves as accountable for their code contributions. Conversely, if clear communication is missing, developers often assume that they can be held accountable for their code contributions. Against this backdrop, we state our first hypothesis:

H1: *In OS software development, software licenses that emphasize legal liability for contributed code (vs. software licenses that do not emphasize legal liability for contributed code) lead developers to perceive higher code accountability.*

Project Integration: Developers' perceived code accountability varies with the integration of OS software projects into other projects, which we determined by measuring the number of forks and stars on GitHub. Accordingly, project integration refers not only to the direct

dependency of other projects on the OS software project but also to the visibility of the OS software project by users, who award the OS software project with a star in recognition of the project. Due to a high level of dependency and visibility, a high degree of integration of the OS software project in other projects or algorithmic systems can be assumed, whereby a defect in the OS software project poses the risk of influencing and impacting numerous other projects and algorithmic systems. Against this backdrop, we observe that developers perceive pressure from the users when contributing to OS software projects that are highly integrated into other projects, compared to OS software projects with fewer integrations. This perceived pressure is reflected in their perceived code accountability; developers tend to evoke higher perceived code accountability when contributing to OS software projects with many integrations into other projects, compared to OS software projects with fewer integrations.

The more the OS software project is integrated into other projects or algorithmic systems, the more developers acknowledge the interest of users in their OS software projects and realize the repercussions of defects in their code contributions, affecting multiple dependent projects and algorithmic systems. Consequently, they perceive themselves as more accountable for ensuring a high quality of their code when contributing to OS software projects characterized by many integrations into other projects, because they put themselves under pressure as they believe users expect a high code quality from them. For this, developers enhance their code quality by reviewing their code more often to prevent defects in dependent projects or algorithmic systems. One developer articulated this viewpoint as follows:

"Because I feel that everything I modify, every bug [defect] I introduce, every mistake I make would affect so many people [users]. I may accidentally break their systems and waste hundreds of hours of other developers just figuring out how to fix this. So that's why I feel more accountable." (OSD16)

Conversely, developers contributing code to OS software projects with few integrations into other projects perceive a lower impact from their OS software projects and, thus, a reduced impact arising from their contributed code. Small OS software projects usually have fewer integrations into other projects since developers often initiate and intend the projects for personal and not public use. This intended use leads developers to be less concerned about the code quality and users

utilizing their code contributions, resulting in lower levels of code accountability perceptions. Additionally, developers contributing to small OS software projects think that users have lower expectations, making them assume less code accountability; thus, they do not perceive the need to accept accountability for their code. In this context, one developer shared with us:

“There are other small things [projects] that I have that are just mainly for myself and that no one else uses. So, I feel more relaxed, just like changing things there. And like, I guess like the accountability is not so much in like, ‘Oh, I’m going to make a mistake that is going to break [something or] is going to leak their data’ or something like that or something super terrible.” (OSD27)

We conclude that project integration plays a pivotal role in shaping developers’ perceived code accountability by indirectly pressuring developers to contribute working code and derive the second hypothesis:

H2: *In OS software development, software projects with high project integration (vs. software projects with low project integration) lead developers to perceive higher code accountability.*

3.3.1.2. Individual-related factors. Work Status: Developers’ work status, which describes whether they contribute code voluntarily in their free time or as employees within organizations receiving payments, affects their perceived code accountability. Voluntary code contributions are associated with lower code accountability perceptions, while contributing code as part of a professional relationship is associated with higher code accountability perceptions.

Developers who contribute code voluntarily in their free time perceive less accountability for their code. They justify their low perception by lacking extrinsic incentives, stating that they invested time without any compensation and that they contributed to OS software projects out of benevolence. In this context, developers are not intrinsically motivated to accept code accountability as they perceive code accountability as an additional burden. They deny taking this additional burden since they are convinced that it is their decision to do whatever they want in their free time. On the contrary, developers believe that users should be pleased that they take the time to contribute code voluntarily. As a compromise, one developer explained to us that users should either pay them for their code contributions to be able to hold the developers accountable or not pay them and not hold them accountable for their code contributions:

“It’s like you buy something to get [your job] faster [done], but it’s free. So, you can’t have everything. You can’t find something free and then ask for accountability to others [...] it’s like a trade-off.” (OSD19)

On the other hand, developers contributing code as part of a professional relationship reported higher code accountability perceptions. Examples of OS software projects that are maintained by organizations and are, therefore, further developed by developers employed by these organizations are *TensorFlow* (Abadi et al., 2015), maintained by Alphabet, and *Presidio* (Microsoft Corporation, 2018), maintained by Microsoft. These employed developers perceive themselves as accountable for their code, as they get extrinsic incentives such as monetary compensation. This compensation leads these developers to perceive higher accountability for their code contributions compared to the code they contribute to OS software projects in their free time. As a result, these developers consider the quality of their code more often and review their code more carefully. In this context, developers explained that they test their code more closely before contributing compared to their free time OS software projects:

“[...] we are aware that some of our clients are using the tools, so [...] we are always testing things before updating the project. So,

probably, I think more about accountability when I do things for a company or when it’s not a personal spare time project.” (OSD21)

Consequently, we assume that developers’ work status shapes their perceived code accountability, with developers contributing code as part of a professional relationship exhibiting higher levels of perceived code accountability compared to those contributing voluntarily in their free time. This assumption aligns with previous IS research that has shown that extrinsic incentives, such as monetary compensation, influence developers’ motivation to contribute to OS software projects (Hertel et al., 2003; Roberts, Hann, & Slaughter, 2006) and demonstrated that developers with voluntary participation show a decreased continued intention to OS software projects (Ho & Rai, 2017). Accordingly, developers perceive themselves as more or less related to the OS software project. However, relatedness is crucial to increase developers’ perceived accountability (O’Donoghue & Van Der Werff, 2022). These findings suggest that extrinsic incentives play a more important role than other incentives, such as intrinsic ones, in accepting accountability for contributed code. We, therefore, assume that within the OS domain, extrinsic incentives such as receiving monetary compensation as part of a professional relationship increase developers’ relatedness to the OS software project and, thus, their perceived accountability for their contributed code. Against this backdrop, we hypothesize the following:

H3: *In OS software development, contributing code to software projects as part of a professional relationship (vs. voluntarily in one’s free time) leads developers to perceive higher code accountability.*

Reputation: Developers’ perceived reputation outside the OS software community affects their perceived code accountability. Developers perceive their code contributions as a means to showcase their development expertise, thus enhancing their career prospects. Accordingly, they highlight the need to prove that they contributed to OS software projects, for example, by collecting badges or showing certificates. Extrinsically motivated by future advantages in career prospects, assuming code accountability helps developers explain and justify their code to others, like future employers. One developer vividly described this as follows:

“I really care about my career, and doing a good job in open source software projects is a good way to showcase my capability in software engineering. And I can always showcase the software I did, I contributed to, I developed to the interviewer while I try to find a job. So that’s why I try to do my best in these open source software projects.” (OSD17)

To fulfill expectations from potential future employers, developers believe that they need to provide high code quality by closely reviewing their code before contributing it. Through these expectations, developers find additional motivation for the activity of reviewing their code as their names are referenced in the OS software project. This motivation leads developers to invest more time and effort in enhancing their code quality by reviewing their code, thereby identifying and correcting defects before contributing their code to the OS software project. In this context, one developer stated:

“The motivation is that you are putting your name on it. So, other people will see how you work. Other people will see how you deal.” (OSD08)

From these insights, we conclude that gaining a reputation extrinsically motivates developers to perceive themselves as accountable for their contributed code. We further base this conclusion on previous IS research that suggested and demonstrated that gaining a reputation motivates developers to add knowledge and increase developers’ performance and involvement in OS software projects (e.g., Wasko, Faraj, & Teigland, 2004; Xu, Jones, & Shao, 2009; Yu, Jiang, & Chan, 2007). Thus, gaining a reputation might increase developers’ extrinsic motivation, which might affect and increase their code accountability perceptions. Accordingly, we derive the following hypothesis:

H4: In OS software development, high reputation (vs. low reputation) for contributing code to software projects leads developers to perceive higher code accountability.

3.3.2. Effects of perceived code accountability on intention to review own code contributions

The perception of code accountability among developers fosters a commitment to contribute high-quality code, expressed by secure and reliable code fulfilling its functions. This emphasis on high code quality primarily arises from high project integrations, their work status, and the pursuit of gaining a reputation. Motivated by these factors, developers are more likely to address and rectify reported defects diligently before contributing their code to an OS software project. Thus, developers put much emphasis on reviewing their code as an important activity to enhance the quality of their code before contributing it to an OS software project. This finding aligns with previous IS research, which underscored the importance of code reviews in providing high-quality code, as developers are more likely to identify and correct defects (Kononenko et al., 2016; McIntosh et al., 2016). As a result, through developers' code review, the quality of an OS software project increases, and users encounter fewer defects and experience more secure and reliable software. Developers succinctly summarized this perspective:

"Because right now, if I'm doing an update and I'm doing some code, I'll push it, I won't think if this is a big security risk. But if you put in the legal obligations [accountability] for that, that would actually improve the quality of the code." (OSD13)

"If I contribute code and I want people to use it, I also want the quality, so I don't want to neglect that, and that's a pressure [through perceived code accountability], so it motivates, yeah." (OSD19)

Furthermore, developers' perceived code accountability strengthens their motivation to increase the quality of their code by reviewing their code and addressing defects they find. Since OS software projects can be misused, for example, by exploiting security vulnerabilities, developers highlight the importance of high code quality to counter such misuse. Against this backdrop, developers emphasize code accountability to assess whether their code in its current form creates more good than bad. Based on this assessment, developers decide whether their code possesses high quality and can be contributed to the OS software project, as one developer told us:

"I won't say it [misuse] is not possible, but I don't think that possibility would stop me or other open source software developers from contributing towards a project that can have more good than bad." (OSD13)

With these insights, we derive the following hypothesis and postulate that developers' perceived code accountability affects their intention to review their code contributions, thereby affecting their intent to conduct activities to increase the quality of their code.

H5: In OS software development, higher perceived code accountability of developers leads them to a higher intention to review their code contributions.

Together with H1-4, we propose that developers' perceived code accountability might not only link the effects of project- and individual-related factors on developers' intention to review their code contributions but also explain these effects. We base this assumption on the qualitative observation that developers repeatedly indicated that project- and individual-related factors affect their behavior within OS software projects by reviewing their code more often. Consequently, we believe that there may be a direct effect of project- and individual-related factors on the intention to review one's own code contributions. Therefore, we also test the research model for a mediation effect of perceived code accountability and state the last hypothesis:

H6: In OS software development, developers' perceived code accountability mediates the effects of (1) legal liability, (2) project integration, (3) work status, and (4) reputation on developers' intention to review their code

contributions.

3.3.3. Study 1: Research model

Our findings demonstrate that project- and individual-related factors significantly impact developers' perceived code accountability, fostering a heightened intention to review their contributions prior to submission. This proactive behavior serves as a key mechanism for maintaining and improving the quality of open-source software projects. Fig. 2 depicts the summary of our findings from the interviews as a research model with the respective hypotheses, which we validate in the next section empirically.

3.4. Study 2: scenario-based factorial survey to validate the research model

3.4.1. Purpose

The qualitative findings from Study 1 served as a foundation for developing our research model (see Fig. 2). However, as Study 1 relied on qualitative statements and developer opinions, the model lacks statistical validity, robustness, and replicability (Nederhof, 1985; Richman, Kiesler, Weisband, & Drasgow, 1999; Venkatesh et al., 2024). To address this limitation, we conducted a scenario-based factorial survey in the second phase of our mixed-method approach (Jasso, 2006; Rossi, 1979; Venkatesh et al., 2024), enabling us to statistically validate the model. This integration of qualitative and quantitative methods strengthened the model's empirical rigor by enhancing its statistical validity, robustness, and replicability (Venkatesh et al., 2024).

4. Method

As mentioned above, to validate our research model, we employed a scenario-based factorial survey, a specialized extension of the scenario method that integrates the strengths of experimental design and survey methods (Rossi, 1979; Vance et al., 2015). Unlike traditional surveys, which are limited in the number of variables they can assess, factorial surveys use vignettes to systematically manipulate and test multiple variables. These vignettes present variations of the same scenario, incorporating different combinations of variables by either showing or hiding them. Participants are then randomly assigned to a subset of these scenarios, asked to immerse themselves in the given context, and subsequently respond to a structured questionnaire (Rossi, 1979).

4.1. Design of the scenario-based factorial survey

To design our scenario-based factorial survey, we developed a scenario that systematically examines the impact of independent variables (*Project Integration*, *Work Status*, *Reputation*, *Legal Liability*) on the dependent variable (*Intention to Review Own Code Contributions*) while also considering the mediating role of *Perceived Code Accountability*. Drawing from insights gained in Study 1 and our overall research endeavor, we selected contributing to an OS software project as our research context. For this, we adapted a GitHub landing page for a fictitious project named *clAIssifier*. This project aimed to develop AI software that autonomously tests and integrates different machine learning classifiers into AI systems. Participants were instructed to imagine they had already contributed code to *clAIssifier*. This framing was essential to ensure participants focused on their intention to review their code contributions rather than first evaluating their willingness to participate in the OS software project. By doing so, we reduced potential external effects that could bias responses.

After creating the scenario, we followed existing IS research and developed UI design artifacts to manipulate our four independent variables (e.g., Adam, 2022; Vance et al., 2015). Thus, we created UI design artifacts to manipulate (1) *Project Integration* (low vs. high), (2) *Work Status* (contributing code voluntarily in one's free time vs. as part of a professional relationship), (3) *Reputation* (low vs. high), and (4) *Legal*

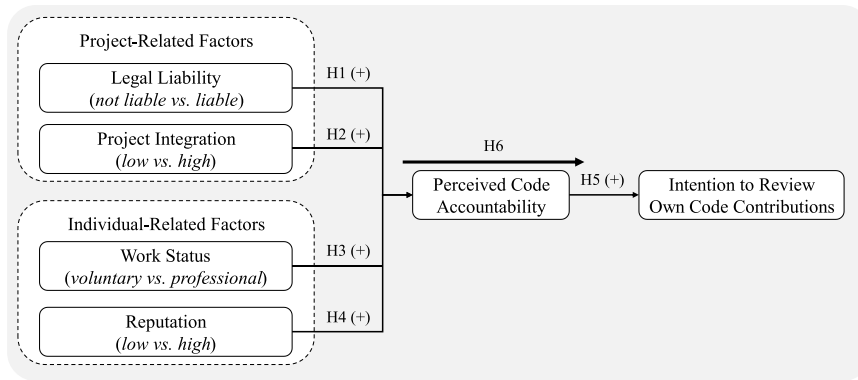


Fig. 2. Research Model.

Liability (absent vs. present) (see Fig. 3 and Fig. 4 in comparison). When developing the UI design artifacts, we were particularly oriented toward existing GitHub features. Hence, we first manipulated *Project Integration* by altering the number of stars and forks of *cIAssifier*, indicating their importance and dependency within the OS community. Second, we depicted *Work Status* by adding a tag to the username reflecting the developers' professional relationship to *cIAssifier*. Third, we used badges to manipulate the *Reputation* that developers earned, aligning with GitHub's practice of awarding achievements for code contributions (Brooks, 2022). And fourth, we based our manipulation of *Legal Liability* on the Apache-2.0 software license, summarizing the statement of this software license regarding being liable or not for code contributions. To enhance the visibility and recognition of the manipulations, we labeled the variables following previous IS research (e.g., Adam, 2022; Vance et al., 2015).

With these four independent variables, we created a 2 (*Project Integration*: low vs. high) x 2 (*Work Status*: contributing code voluntarily in one's free time vs. as part of a professional relationship) x 2 (*Reputation*: low vs. high) x 2 (*Legal Liability*: absent vs. present) design, resulting in 16 different vignette variations for the scenario-based factorial survey. Each participant randomly received four different vignettes and subsequently answered questions about their *Perceived Code Accountability* and their *Intention to Review Own Code Contributions*.

After developing our UI design artifacts, we structured the

questionnaire for the scenario-based factorial survey, which included socio-demographic items, items about participants' personal experience with OS software projects, pre-vignette items, and post-vignette items (see Fig. 5). Upon entering the scenario-based factorial survey, participants first answered questions about their socio-demographics and personal experience with OS software projects. Next, they responded to pre-vignette items assessing their general *Perceived Code Accountability* within OS software projects. Afterward, we introduced participants to the scenario before they were presented with the first vignette. For each vignette, participants answered post-vignette items, first evaluating their *Intention to Review Own Code Contributions* and then their *Perceived Code Accountability*. After finishing answering the post-vignette items, we showed the participants the second vignette and repeated this process another three times, whereby each participant rated four random vignettes. When the participants finished evaluating four different vignettes, we finally asked them to assess the realism of the scenario to control for external validity (Lapierre, 2007).

4.1.1. Construct measurement for mediator and dependent variable

We adapted established scales from previous research to the OS domain to measure the mediator *Perceived Code Accountability* and the dependent variable *Intention to Review Own Code Contributions*. Specifically, we adapted established scales for *Perceived Code Accountability* (Hochwarter, Ferris, Gavin, & Perrew, 2007) and *Intention to Review Own*

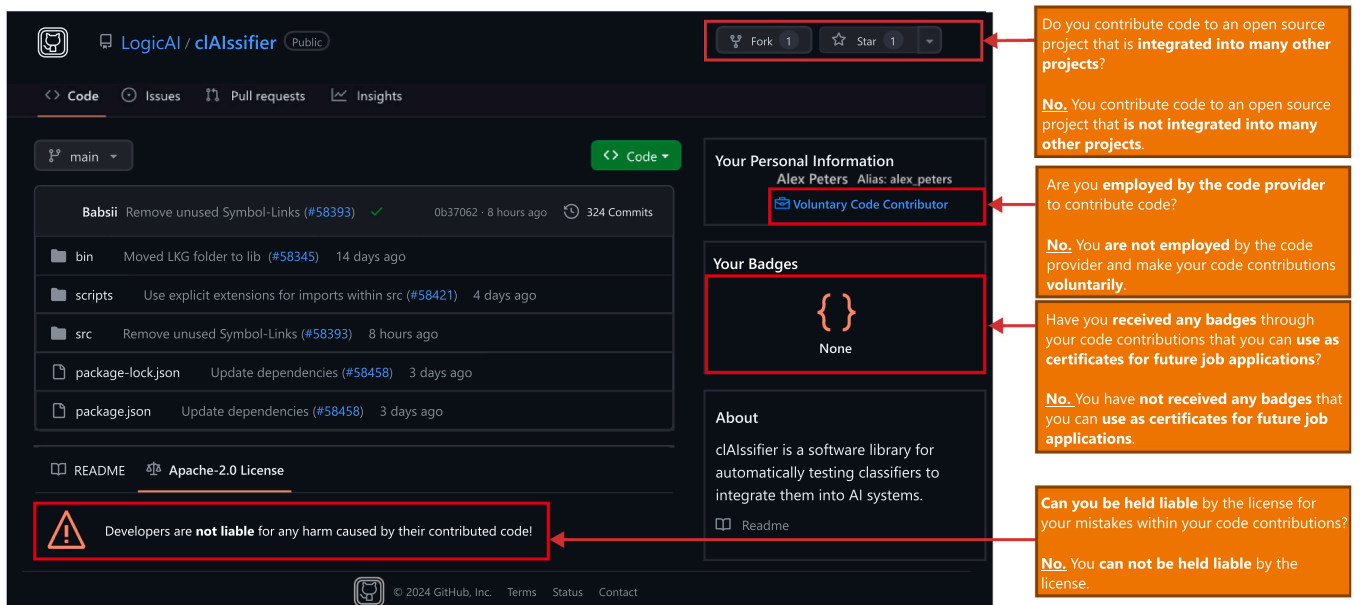


Fig. 3. Example of the Vignettes Used (Project Integration: Low, Work Status: Contributing Code Voluntarily, Reputation: Low, and Legal Liability: Present).

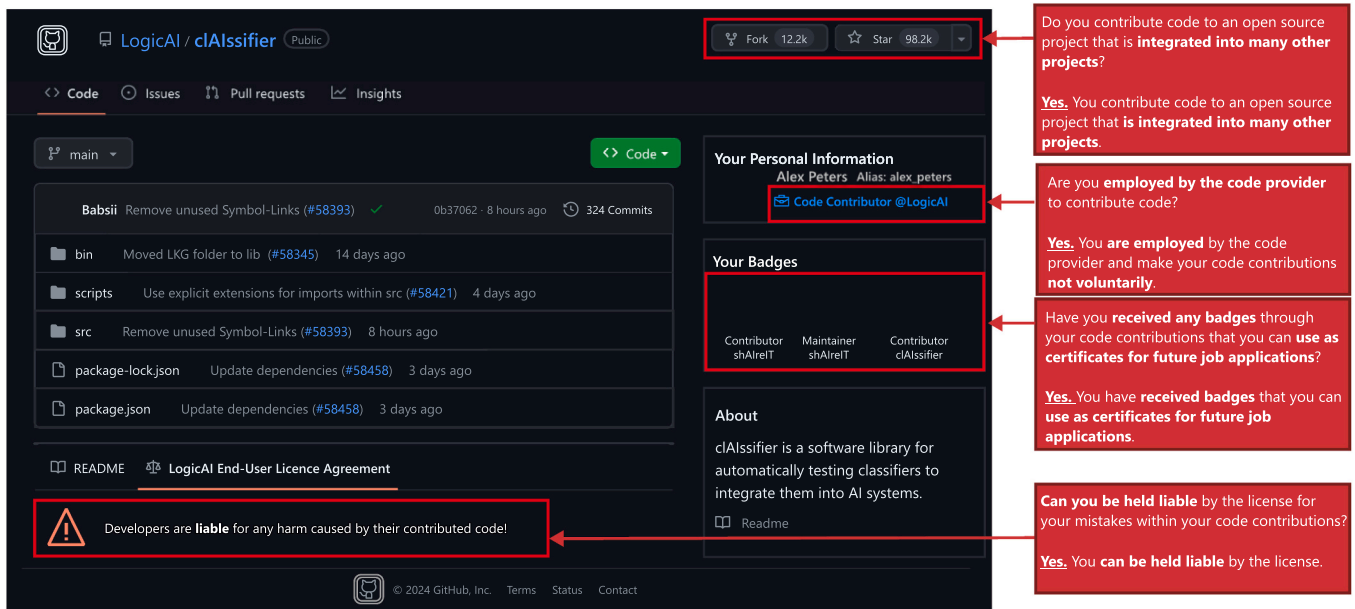


Fig. 4. Example of the Vignettes Used (Project Integration: High, Work Status: Contributing Code as Part of a Professional Relationship, Reputation: High, and Legal Liability: Absent).

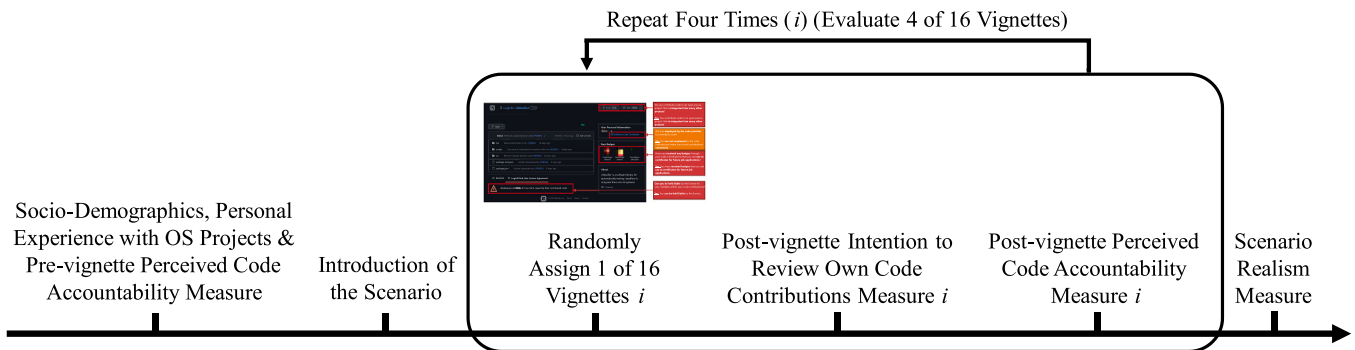


Fig. 5. Sequence of the Scenario-Based Factorial Survey.

Code Contributions (Sykes, Venkatesh, & Gosain, 2009), measured on a 7-point Likert-type scale ranging from *strongly disagree* (1) to *strongly agree* (7). To enhance the robustness of our research model, we included several control variables to test it for alternative hypotheses (Spector & Brannick, 2011). For this purpose, we collected *Age*, *Gender*, and *Education* as classical control variables. Moreover, we collected the *Number of Participated OS Software Projects* and the *Number of Code Commits to OS Software Projects*, as we assumed that participants with more OS software project participation or code commits would have a higher *Perceived Code Accountability* due to increased responsibilities within previously participated OS software projects (e.g., Mahmud, Islam, Luo, & Mikalef, 2024). Additionally, to ensure external validity, we included items to assess the realism of the scenario and also measured them with the same 7-point Likert-type scale (Lapierre, 2007). Table 9 in the Appendix summarizes the constructs and their respective measurements used for the scenario-based factorial survey.

4.1.2. Data collection and participant demographics

Before collecting our data, we tested our scenario-based factorial survey through three interviews (lasting between 35 and 45 min) with experts in the OS domain to ensure clarity and refinement of the constructs and items. The provided feedback improved the quality of our questionnaire iteratively. After improving our questionnaire, we used the crowdsourcing platform *Prolific*, which is known for its high data

quality, to survey 145 OS developers, evaluating a total of 580 vignettes (Palan & Schitter, 2018).

Since *Prolific* does not provide any specific filter criteria for OS developers, we first conducted a prescreening study to identify developers who contribute to OS software projects, are at least 18 years old, and are U.S. citizens. For this, we initially set a filter to get participants who were at least 18 years old and U.S. citizens. We then subsequently asked 225 participants to name an OS software project to which they contributed code, as well as the approximate number of code commits to the OS software project. To ensure participants' anonymity, we did not specifically ask them for their usernames to manually check their code contributions to the OS software project mentioned. Instead, we relied on the information provided by the developers. This procedure ensured that we could not reference any data and, therefore, could not identify any OS developer. After collecting the prescreening data, we filtered out participants who indicated that they had not contributed at least one code commit to OS software projects and filtered out participants who did not specify any traceable OS software projects, such as reference to OS software projects like 'Microsoft,' 'Lenovo,' and 'Arduino-based spy cam.' The prescreening study gave us a number of trustworthy OS developers, which we then asked to participate in our scenario-based factorial survey. Moreover, these filter criteria ensured that the participants could understand our questionnaire and formed a homogeneous group regarding culture and legal regulations, thereby reducing external

effects on our scenario-based factorial survey.

When conducting the prescreening study and the scenario-based factorial survey, we emphasized that participation in these studies is voluntary and that the participants can withdraw from the studies at any time. We also emphasized that the responses would not affect them personally and assured them an anonymous and secure data transmission, adhering to practices from previous IS research to mitigate social desirability bias (e.g., Gwebu, Wang, & Hu, 2020).

After conducting the scenario-based factorial survey, we processed the data by filtering participants using instructional manipulation checks (deleted 12 data points) and nonsensical questions (deleted 36 data points) that served as attention checks (Oppenheimer, Meyvis, & Davidenko, 2009; Paolacci, Chandler, & Ipeirotis, 2010). We also excluded vignettes where participants did not pay much attention to the vignettes and failed subsequent manipulation checks (deleted 52 data points). Within these manipulation checks, we asked the participants which characteristics of the independent variable they saw to ensure that the evaluation of the vignette was based on the actual presented characteristics. This data processing resulted in 480 evaluated vignettes, providing high-quality data for the data analysis.

Lastly, the socio-demographic analysis reveals a heterogeneous sample with 68.40 % male, 26.30 % female, and 5.30 % identifying as non-binary. The average age was 37.05 years ($SD = 12.58$ years), with 82.00 % holding a bachelor's degree or higher. Finally, the participants stated that they contributed code to an average of 4.92 OS software projects ($SD = 7.24$), with 49.60 % having ten or more code commits.

4.1.3. Measurement validation

To test the validity of the measurement model, we performed a confirmatory factor analysis (see Table 10 and Table 11 in the appendix). First, we examined the indicator loadings of the measured constructs, which exceeded the necessary threshold of 0.71 (Hair et al., 2021). We then tested the constructs for internal consistency using Cronbach's Alpha and ρ_C as composite reliability criteria. Both values were above 0.70, indicating high internal consistency and composite reliability (Hair et al., 2021). Next, we assessed the convergent validity of the measurement model by calculating the average variance extracted (AVE) values. With all values exceeding 0.50, the constructs demonstrated high convergent validity (Hair et al., 2021). Finally, to ensure the measurement models' discriminant validity, we used the heterotrait-monotrait ratio of correlations (HTMT) values, which are more robust than the Fornell-Larcker criterion (Hair et al., 2021; Radomir & Moisescu, 2020). The analysis of the HTMT values retrieved the largest value of 0.64, well below the critical threshold of 0.85 (Henseler, Ringle, & Sarstedt, 2015). Nevertheless, as the Fornell-Larcker criterion is widely used within IS research (e.g., Gwebu et al., 2020; Vance et al., 2015), we also calculated it in an additional analysis and found no issues regarding discriminant validity (see Table 12 in the appendix). Consequently, both the HTMT and Fornell-Larcker criterion supported the measurement models' high discriminant validity.

After successfully conducting the confirmatory factor analysis, we checked our data for common method biases, sufficient statistical power, and the realism of the scenario rated by the participants. Therefore, we followed previous IS research (James, Wallace, & Deane, 2019) and first performed Harman's single-factor test before calculating the construct correlation matrix. Harman's single-factor test showed a value of 26.33 %, well below the threshold of 50.00 %, indicating that no single factor accounts for more than 50.00 % of the total variance in the data (Fuller, Simmering, Atinc, Atinc, & Babin, 2016). Additionally, the highest value in the construct correlation matrix was 0.59, which was well below the threshold of 0.90 (James et al., 2019). Thus, common method bias was unlikely to affect the results (James et al., 2019). Next, we tested the data to determine whether the sample size of 480 vignettes was sufficiently large to support our results statistically. For this, we performed a post-hoc power analysis using *G*Power*. The analysis

yielded a power of 0.90, which exceeded the conventional threshold of 0.80 ($\alpha = 0.05$; $\beta = 0.20$), indicating that our sample size is adequate to support the results statistically (Cohen, 1992; Ellis, 2010; Faul, Erdfelder, Buchner, & Lang, 2009). Finally, to ensure external validity, we evaluated the realism of the scenario used in our scenario-based factorial survey. Participants rated the realism of the scenario with a mean value of 5.91 ($SD = 1.03$) on a 7-point Likert-type scale ranging from *strongly disagree* (1) to *strongly agree* (7), suggesting that our utilized scenario was perceived as realistic. We, therefore, conclude that our scenario is likely to appear in real-world situations and, thus, supports external validity (Schmidt, Bartsch, & Adam, 2023).

4.2. Study 2: results

We performed a hierarchical generalized mixed model analysis to validate our research model. We decided to use the generalized mixed model analysis over ordinary least squares because participants in our scenario-based factorial survey received and evaluated four different vignettes, resulting in non-independent measurements of the constructs (Piquero, Lyn Exum, & Simpson, 2005). Generalized mixed models overcome the non-independence of the constructs and allow us to incorporate fixed and random effects (McLean, Sanders, & Stroup, 1991). For this purpose, we used the R-package *lme4* (Bates, Mächler, Bolker, & Walker, 2015) to validate our research model and coded the independent variables (i.e., *Project Integration*, *Work Status*, *Reputation*, and *Legal Liability*) as binary variables. Hence, we coded *Project Integration* (low vs. high), *Work Status* (contributing code voluntarily in one's free time vs. as part of a professional relationship), *Reputation* (low vs. high), and *Legal Liability* (absent vs. present) as 0 vs. 1. In addition, we derived and created the construct *Delta Perceived Code Accountability* by subtracting the pre-vignette measurements of *Perceived Code Accountability* from the post-vignette measurements of *Perceived Code Accountability*. Subtracting the pre-vignette from the post-vignette measurements was crucial for the subsequent analysis, as we focused on the change in *Perceived Code Accountability* due to the presented independent variables rather than the absolute value.

4.2.1. Testing the main effects model

We divided the hierarchical generalized mixed model analysis into five steps (see Table 4 and Table 5). First, we tested the effects of the collected control variables on the mediator, *Delta Perceived Code Accountability* (Model 1). Second, we added the independent variables to the model and measured their effects on the mediator, *Delta Perceived Code Accountability* (Model 2). Third, we tested the effects of the collected control variables on the dependent variable, *Intention to Review Own Code Contributions* (Model 3). Fourth, we added the independent variables to the model and measured their effects on the dependent variable, *Intention to Review Own Code Contributions* (Model 4). Fifth, we added the mediator *Delta Perceived Code Accountability* to the model and measured the effects on the dependent variable, *Intention to Review Own Code Contributions* (Model 5).

The results in Table 4 indicate that *Project Integration* ($\beta = 0.40$, $p < 0.001$), *Work Status* ($\beta = 0.64$, $p < 0.001$), *Reputation* ($\beta = 0.44$, $p < 0.001$), and *Legal Liability* ($\beta = 0.59$, $p < 0.001$) significantly affect *Delta Perceived Code Accountability* in the intended direction. Through these significant effects, we find support for H1, namely that software licenses that emphasize legal liability for contributed code (vs. software licenses that do not emphasize legal liability for contributed code) lead developers to perceive higher code accountability. Additionally, the results indicate that OS software projects with high project integration (vs. OS software projects with low project integration) lead developers to perceive higher code accountability, supporting H2. Moreover, the significant effect of *Work Status* on *Delta Perceived Code Accountability* implies that contributing code to OS software projects as part of a professional relationship (vs. voluntarily in one's free time) leads developers to perceive higher code accountability, thus supporting H3.

Table 4
Effects of the Independent Variables on Delta Perceived Code Accountability.

	Model 1		Model 2	
	Coeff.	SD	Coeff.	SD
Intercept	−0.29	1.00	−1.73	0.98
Manipulation				
Project Integration			0.40***	0.09
Work Status			0.64***	0.09
Reputation			0.44***	0.10
Legal Liability			0.59***	0.10
Controls				
Age	−0.01	0.01	−0.01	0.01
Gender	−0.07	0.19	−0.07	0.18
Education	−0.00	0.17	0.08	0.19
Number of Participated OS Software Projects	0.01	0.01	0.01	0.01
Number of Code Commits to OS Software Projects	0.25	0.13	0.25*	0.12
Marginal R ²	0.03		0.16	
Conditional R ²	0.45		0.58	
AIC	1652		1565	

Note: *p < 0.05

**p < 0.01

***p < 0.001

n = 480

Table 5
Effects of Delta Perceived Code Accountability on Intention to Review Own Code Contributions.

	Model 3		Model 4		Model 5	
	Coeff.	SD	Coeff.	SD	Coeff.	SD
Intercept	5.19***	0.86	4.32***	0.85	5.22	0.77
Manipulation						
Project Integration			0.24**	0.09	0.03	0.07
Work Status			0.38***	0.09	0.05	0.07
Reputation			0.31***	0.09	0.08	0.07
Legal Liability			0.30***	0.09	−0.01	0.07
Mediator						
Delta Perceived Code Accountability					0.52***	0.03
Controls						
Age	−0.01	0.01	−0.01	0.01	−0.00	0.01
Gender	−0.11	0.16	−0.11	0.16	−0.08	0.15
Education	0.19	0.15	0.24	0.15	0.20	0.13
Number of Participated OS Software Projects	0.01	0.01	0.01	0.01	0.01	0.01
Number of Code Commits to OS Software Projects	0.10	0.11	0.11	0.11	−0.02	0.10
Marginal R ²	0.03		0.09		0.36	
Conditional R ²	0.46		0.51		0.71	
AIC	1492		1464		1286	

Note: *p < 0.05

**p < 0.01

***p < 0.001

n = 480

Finally, we find **support for H4**, namely that having a high reputation (vs. low reputation) for contributing code to OS software projects leads developers to perceive higher code accountability. Among these significant effects, *Work Status* and *Legal Liability* have the strongest effects on *Delta Perceived Code Accountability*, while *Reputation* and *Project Integration* have the lowest effects.

Next, the results in **Table 5** show significant direct effects of *Project Integration* ($\beta = 0.24$, $p < 0.01$), *Work Status* ($\beta = 0.38$, $p < 0.001$), *Reputation* ($\beta = 0.31$, $p < 0.001$), and *Legal Liability* ($\beta = 0.30$, $p < 0.001$) on the dependent variable, *Intention to Review Own Code Contributions*. Additionally, the results indicate that *Delta Perceived Code Accountability* significantly affects the *Intention to Review Own Code Contributions* ($\beta = 0.52$, $p < 0.001$), **supporting H5**. Thus, higher levels

of perceived code accountability lead developers to a higher intention to review their code contributions.

4.2.2. Testing for mediation

To test the potential mediation effects of *Delta Perceived Code Accountability* (H6), we followed previous IS research and performed a bootstrap analysis with 5000 samples to calculate confidence intervals and assess the mediation effects (Hayes, 2009; Vance et al., 2015). For this, we multiplied *Path a* (*Independent Variable* toward *Delta Perceived Code Accountability*) with *Path b* (*Delta Perceived Code Accountability* toward *Intention to Review Own Code Contributions*) and calculated *Path c* (*Independent Variable* toward *Intention to Review Own Code Contributions*). When assessing the mediation effects, we find support for a full mediation if the confidence intervals of *Path a*b* and *Path c* do not include zero. However, when the confidence intervals of *Path a*b* do not include zero, but the confidence interval of *Path c* includes zero, we find support for a partial mediation (MacKinnon, 2012; Shrout & Bolger, 2002).

Table 6 shows the results of the mediation analysis. Since the confidence intervals for both *Path a*b* and *Path c* did not include zero, *Delta Perceived Code Accountability* fully mediates the effects of the independent variables toward the *Intention to Review Own Code Contributions*, **supporting H6**. We, therefore, conclude that developers' perceived code accountability mediates the effects of project integration, work status, reputation, and legal liability on developers' intention to review their code contributions.

4.3. Synthesis of qualitative and quantitative study results

The synthesis of our qualitative and quantitative results offers a deep and holistic understanding of how developers' perceived code accountability emerges in the OS domain, where formal consequences are typically absent or only weakly pronounced, and how this perception impacts the code quality of developers' contributed code. By adopting a developmental mixed-method research approach (Venkatesh et al., 2024), we combined insights from qualitative interviews and existing literature to derive a research model, which we subsequently tested using a scenario-based factorial survey. This approach allowed us to connect individual perspectives with empirical validation, enhancing both the theoretical contributions and practical implications of our paper.

Our qualitative study identified that developers' perceived code accountability stems from project- and individual-related factors. Among project-related factors, the recognition and understanding of legal liability for contributed code and the level of project integration emerged as key factors in shaping developers' perceived code accountability. We found that developers often lack awareness and clear communication of software license implications, as these licenses are frequently not emphasized within OS software projects. Consequently, many developers are unaware that they do not bear any legal liability for their code contributions. This lack of understanding leads developers to perceive themselves as accountable for their code, believing they must explain and justify their code, particularly if their contributed code causes harm. Thus, clear communication about the legal liabilities of software licenses is essential to shape developers' perceived code accountability. Our quantitative analysis supports this finding, demonstrating that improving communication about legal liability significantly enhances developers' perceived code accountability (H1). Additionally, our qualitative study revealed that developers view the integration and dependency of an OS software project within other projects or algorithmic systems as a crucial factor regarding their perceived code accountability. Developers explained that as the integration and dependency of an OS software project increase, so does its potential impact. This heightened impact makes developers perceive pressure from users and makes them more aware of the far-reaching consequences of their potentially faulty code, thereby increasing their code accountability perceptions. Quantitative validation of this insight

Table 6

Results of the Bootstrapped Confidence Interval Tests for Mediation.

Independent Variable	Path a*b			Path c			Type of Mediation
	2.5 % lower bound	97.5 % upper bound	Zero included?	2.5 % lower bound	97.5 % upper bound	Zero included?	
Project Integration	0.12	0.31	No	0.08	0.42	No	Full
Work Status	0.24	0.44	No	0.21	0.55	No	Full
Reputation	0.14	0.34	No	0.14	0.48	No	Full
Legal Liability	0.21	0.42	No	0.13	0.47	No	Full

provided statistical confirmation, strengthening the robustness of this finding (H2).

Individual-related factors, such as contributing code to OS software projects as part of a professional relationship and the pursuit of reputation gains, play crucial roles in shaping developers' perceived code accountability. In our qualitative study, developers contributing through professional roles reported a stronger awareness of code accountability compared to those contributing voluntarily. Developers explain their heightened code accountability perception through extrinsic motivations such as the monetary compensation associated with their professional role (H3). Finally, developers highlighted the importance of reputation as an extrinsic motivator, explaining that their code accountability perceptions are closely tied to their desire to take ownership of their code contributions, particularly to enhance their career prospects beyond the OS community (H4). Our quantitative analysis supports both findings, validating the influence of professional relationships and reputation as key drivers of developers' perceived code accountability.

Furthermore, our findings underscore the behavioral implications of developers' perceived code accountability. In our qualitative study, developers reported that their code accountability perception drives their intention to rigorously review their code to ensure contributing code with high quality (H5). This observation is further supported by our quantitative analysis, which demonstrates that perceived code accountability explains the influence of legal liability, project integration, work status, and reputation on developers' intention to review their code (H6). Thus, both project-related and individual-related factors not only shape developers' perceived code accountability but also serve as critical motivators for encouraging thorough code review before developers contribute code to OS software projects.

In summary, the synthesis of our qualitative and quantitative findings, presented in Table 7, highlights the critical role of project- and individual-related factors to evoke developers' perceived code accountability and to increase their intention to review their code before contributing it to OS software projects. Our mixed-method research approach not only validates the research model developed from qualitative insights and existing literature but also enhances its generalizability through statistical rigor. Together, these findings provide a comprehensive understanding of how perceived code accountability can be cultivated in the OS domain by leveraging project- and individual-related factors. As a result, our paper contributes to the theoretical discourse on accountability in the OS domain and offers practical strategies for improving code quality in OS software projects.

5. Discussion

The widespread use of OS software in algorithmic systems underscores the need for secure and reliable OS software projects, regardless of how high code quality is ensured. Against this backdrop, this paper examines the factors shaping developers' perceived code accountability in the OS domain and how these perceptions influence their intention to review their code to support high-quality code contributions. Our qualitative and quantitative findings indicate that project integration (i.e., contributing to highly integrated OS software projects), work status (i.e., contributing as part of a professional role), reputation (i.e., gaining recognition within the OS community), and

legal liability (i.e., contributing under licenses that hold developers accountable) increase developers' perceived code accountability, leading to a higher intention to review own code contributions. Among these factors, work status and legal liability had the strongest effects, whereas reputation and project integration showed comparatively weaker influences. Overall, our findings suggest that perceived code accountability can be fostered in the OS domain in which formal consequences are absent or only weakly pronounced, motivating developers to engage in reviewing their own code contributions and thereby improving code quality in OS software projects.

5.1. Contributions to research

This paper advances IS research on accountability in OS software development, in four important ways: First, we demonstrate that in OS software development, code accountability perceptions can emerge through a complex interplay of project- and individual-related factors, even when formal consequences are absent or only weakly pronounced. While prior IS research has emphasized the importance of code accountability, it has primarily examined contexts where formal consequences enforce it, offering limited insights into the OS software context in which such mechanisms are absent or only weakly pronounced to foster accountability (e.g., Ahmed & van den Hoven, 2010; Martin, 2019). Our findings challenge the prevailing assumption that accountability requires enforceable formal consequences (Bovens, 2007; Novelli et al., 2023). By identifying additional factors that shape developers' perceived code accountability, we broaden the understanding of accountability perceptions in the OS software development beyond traditional enforceable and formal consequences. Therefore, these additional factors must not be exclusively limited to the development of OS software as they can also inform general software engineering practices in professional, non-OS environments where formal consequences exist but may be insufficient to ensure code contributions with high-quality. Our insights thus offer a transferable framework for fostering code accountability perceptions in other development contexts such as corporate software teams. This insight encourages IS researchers to consider a broader set of factors as possible drivers of code accountability perceptions, expanding IS research perspectives on developer behavior and governance.

Second, we establish perceived code accountability as a key motivator for developers to review their code before contributing to OS software projects, thereby enhancing overall code quality. Prior IS research has largely focused on increasing the number of contributions while paying limited attention to mechanisms that ensure code contributions with high quality (e.g., Hornuf & Jeworrek, 2023; McIntosh et al., 2016; Krogh et al., 2012). By demonstrating that perceived code accountability drives engagement in quality-improving activities, we highlight its pivotal role in shaping beneficial developer behavior. Notably, perceived code accountability can motivate developers to undertake less favored yet crucial tasks, such as self-reviewing code, which directly improves the security and reliability of OS software projects. This positions code accountability perceptions as a behavioral mediator between environmental and individual characteristics and quality outcomes. As a result, fostering perceived code accountability provides a key strategy for improving the code quality of OS software projects, suggesting that future IS research should incorporate code

Table 7
Mapped Results of Qualitative and Quantitative Studies.

Qualitative Study	Hypotheses	Quantitative Study	Combined Interpretation
The interviews suggest that the communication, recognition, and understanding of software licenses in terms of their legal liabilities affect developers' perceived code accountability.	H1: In OS software development, software licenses that emphasize legal liability for contributed code (vs. software licenses that do not emphasize legal liability for contributed code) lead developers to perceive higher code accountability.	Supported	Developers perceive themselves as less accountable for their contributed code when they are aware that software licenses protect them against any legal liability that arises from their contributed code.
The interviews suggest that the integration of the OS software projects into other projects or algorithmic systems influences developers' perceived code accountability, as their code contributions can have far-reaching impacts across dependent projects and various algorithmic systems.	H2: In OS software development, software projects with high project integration (vs. software projects with low project integration) lead developers to perceive higher code accountability.	Supported	Developers recognize the impact of their code contributions and take accountability for not negatively impacting other projects or algorithmic systems.
The interviews suggest that contributing code to OS software projects as part of a professional relationship affects developers' perceived code accountability because of extrinsic motivations such as the compensation of their employers.	H3: In OS software development, contributing code to software projects as part of a professional relationship (vs. voluntarily in one's free time) leads developers to perceive higher code accountability.	Supported	Developers' perceived code accountability is influenced by whether they get compensated for their code contributions or not.
The interviews suggest that earning a reputation for code contributions to OS software projects extrinsically motivates developers to take ownership of their code contributions and, thus, perceive themselves accountable for them.	H4: In OS software development, high reputation (vs. low reputation) for contributing code to software projects leads developers to perceive higher code accountability.	Supported	Developers are motivated to assume code accountability by the opportunity to gain a reputation and increase their career prospects in future job applications.
The interviews suggest that developers who perceive themselves as accountable for their code are more motivated to review their code before contributing it, resulting in fewer defects and, thus, higher code quality.	H5: In OS software development, higher perceived code accountability of developers leads them to a higher intention to review their code contributions.	Supported	Developers' perceived code accountability enhances their motivation to take on the less favored task of reviewing their code before contributing it to OS software projects.
The interviews suggest that legal liability, project integration, work status, and reputation affect not only their perceived code accountability but also their intention to review their code contributions to enhance the quality of their contributed code.	H6: In OS software development, developers' perceived code accountability mediates the effects of (1) legal liability, (2) project integration, (3) work status, and (4) reputation on developers' intention to review their code contributions.	Supported	Developers' perceived code accountability can be evoked by legal liability, project integration, work status, and reputation to motivate developers to review their code before contributing it.

accountability perceptions as a latent driver of quality-oriented efforts.

Third, and from a broader perspective, we position perceived code accountability as a fundamental yet underexplored control mechanism within OS governance, thereby enhancing code quality beyond reliance on traditional structural governance mechanisms. While prior IS research has primarily focused on formal governance structures such as coordination and organization (e.g., Krogh et al., 2012), it has largely overlooked how code accountability perceptions influence the code quality in OS software projects. Our findings reveal that informal consequences, such as reputation-based incentives and project integration, serve as effective substitutes for formal enforcement mechanisms, extending OS governance beyond relying on explicit guidelines and rules. As a result, we advocate for OS governance frameworks that deliberately foster informal consequences, such as reputation-based incentives, to enhance code quality. By doing so, our research fills a critical gap in IS research on OS governance, specifically addressing how to increase and ensure high code quality by providing actionable strategies for OS governance (Krogh et al., 2012). Filling this gap not only advances research on OS governance but also provides insights for similar distributed, non-hierarchical software development environments to ensure high code quality.

Finally, and more broadly, we provide context-specific insights from OS software development that advance accountability theory in IS research (Vance et al., 2013). Our context-specific insights show that the established antecedents of perceived accountability manifest differently in decentralized software development environments. Specifically, we find that the mechanism of 'identifiability' from accountability theory relates to visible contributions to widely integrated projects, and reputation building relates to the 'expectation of evaluation' and 'awareness of monitoring', while we find no equivalent for contextualizing 'social presence'. This finding highlights the boundary conditions of accountability theory in OS software development. However, we also identify legal liability and work status as additional antecedents that shape accountability perceptions in open, collaborative software development

environments, revealing antecedents that have not yet been theorized in prior IS research. Moreover, we broaden the scope of accountability theory by demonstrating that accountability perceptions not only deter undesirable behavior but also foster positive discretionary practices, such as voluntary code self-review and proactive quality improvement, which underscores accountability perceptions as a driver of both compliance and constructive engagement. Taken together, these context-specific insights from OS software development suggest an expanded accountability theory that captures both compliance and quality-enhancing behavior, thereby offering pathways toward a more general theory of governance and control in decentralized software development environments.

5.2. Implications for practice

Our findings offer actionable insights for maintainers of OS software projects, particularly those unable to rely on formal consequences to enforce code accountability yet seeking to improve code quality. By leveraging informal consequences, maintainers can integrate functionalities inspired by our developed UI design artifacts into the structure of their OS governance. In particular, maintainers can integrate the shown UI design artifacts in their project management tools, such as GitHub, to influence developers' recognition of legal liability, project integration, work status, and reputation. In particular, to enhance the impact of legal liability, maintainers could obscure the legal disclaimers that absolve developers of legal liability, preventing developers from immediately recognizing and relying on these clauses to avoid accountability for their code contributions. This subtle design change encourages developers to act as though they remain accountable for their code and, thus, be motivated to review their contributed code.

Second, maintainers can underscore their project's integration by highlighting its dependencies within other projects or algorithmic systems. For instance, frequently referencing the number of forks or examples of past vulnerabilities caused by defects in OS software projects

(e.g., *Log4j*, *libssh*, and *Apache Struts 2*) within their project communications can illustrate the critical need for thorough code reviews. These communications can emphasize how individual code contributions directly affect the reliability of interconnected projects and algorithmic systems, reinforcing the importance of contributing code with high quality.

Third, to leverage work status, maintainers should consider seeking funding from organizations that benefit from the OS software projects to employ developers or provide monetary compensation for their voluntary contributions. Collaborations with organizations can also help secure funding to support contributors directly. Additionally, registering OS software projects on bounty platforms like GitPay ([Gitpay, 2024](#)) or Huntr ([Huntr, 2024](#)) allows users to compensate developers for specific features or fixed defects, increasing developers' accountability perceptions for their code contributions.

Fourth, maintainers can create incentive systems to reward high-quality code contributions and thus leverage the effects of increased reputation. For example, developers could be recognized based on metrics such as the number of issues referencing their code or feedback from peers. Upon meeting predefined quality standards, maintainers could issue certificates or digital badges that developers can showcase in their professional portfolios or job applications, enhancing their reputation beyond the OS community.

While integrating these practices in OS software projects can increase developers' perceived code accountability and improve code quality, maintainers should recognize the trade-off between higher code quality and project progress. High-quality code contributions often require greater effort and time, which could slow the pace of development. Thus, we recommend that OS software project maintainers find a balance within their OS governance strategies to encourage high-quality code contributions while maintaining the agility needed for rapid feature development.

5.3. Limitations and directions for future research

Our research is subject to certain limitations that pave the way for future research: First, we developed our research model primarily by interviewing developers who identified themselves as male. Although 95 % of developers contributing to OS software projects identify themselves as male, we acknowledge a gender imbalance in our online interviews ([Filippova et al., 2017](#)). Future research should strive for a more balanced sample by including more developers who identify themselves as female or non-binary. A more balanced sample would enhance the robustness of future research, allowing for a better understanding of differences in perceived code accountability based on gender identity and reducing potential biases arising from gender identity.

Second, our developmental mixed-method research approach relied on online interviews and a scenario-based factorial survey. Thus, we tested our developed research model using an online experiment, but we were limited in testing the developed research model in real practice. Future research could conduct field experiments or quasi-experiments using publicly available data from OS software projects to test the developed research model in real practice. Drawing on other research approaches would also allow the application of advanced statistical techniques such as hierarchical linear modeling or alternative multilevel analyses. Such analyses would provide valuable insights into the dependencies and interaction effects for each factor in order to understand individual and project-related factors not only from a micro but also from a macro level. Additionally, such research could also go beyond developers' intention to review their code to investigate the effects of developers' perceived code accountability on reviewing their code by measuring developers' actual behavior.

Third, while our research provides generalized insights by interviewing and surveying developers from diverse OS software projects, it lacked an in-depth exploration of specific OS software projects. Conducting case studies or action research on particular OS software

projects could uncover deeper insights into the code accountability dynamics within those OS software environments. Such focused studies would be crucial in understanding internal practices shaping developers' code accountability perceptions and provide more insights into the integration of perceived code accountability into the governance of OS software projects. In particular, such studies might explore additional factors affecting developers' perceived code accountability that go beyond the identified and statistically tested factors of legal liability, project integration, work status, and reputation, and include potential moderation effects such as social dynamics like peer influence and team dynamics.

Fourth, our studies concentrated on developers' code accountability perceptions in the context of OS software development and did not include a comparative analysis with other settings, such as proprietary software development. We acknowledge that accountability mechanisms are also present in other software development contexts, and a systematic cross-contextual comparison could provide valuable insights into the relative importance of these mechanisms within each setting. Since such an analysis was beyond the scope of our research, we encourage future research to address this gap and advance the understanding of the similarities, differences, and potential transferability of accountability mechanisms across diverse software development environments.

Fifth, we concentrated on the developers' perspective and did not delve into how organizations manage the lack of holding developers of OS software projects accountable for flawed OS software projects that negatively affect their algorithmic systems. Thus, future research should investigate how organizations address and compensate for the absence of code accountability within OS software projects. Such research would provide a nuanced view of why organizations continue to use OS software projects despite being accountable for their implemented flawed algorithmic systems (e.g., [Council of the European Union, 2024](#); [Schnaack & Termer, 2023](#)). Additionally, it would offer insights into effective management and adaptation strategies for organizations dealing with absent code accountability of OS software projects, helping organizations mitigate the risks of using OS software projects more effectively.

Sixth, and finally, the landscape of OS software development is undergoing significant transformations, with increasing organizational participation ([Bubb, 2025](#)). A growing number of organizations invest in OS software projects to gain strategic advantages, such as "shorten development cycles, reduce costs, promote innovation and talent, and improve software quality" ([Porsche, 2023](#)). This trend not only facilitates collaboration among industry peers but also formalizes code contributions of OS software projects as organizations integrate OS engagement into employees' professional responsibilities to expand their technological reach by broader adoption ([Whiting, 2022](#)). Reflecting this shift, over 30 % of Fortune 100 companies had formal programs to support or maintain OS software projects in 2022, with participation continuing to rise ([Wolf, 2022](#)). However, this increasing institutionalization of organizations in OS software projects raises concerns about the long-term relevance of informal consequences to improve code quality. As organizations assume a larger role in OS software projects, organizational incentives and performance metrics may gain more importance, potentially diminishing the influence of informal consequences. Consequently, our findings, which primarily focus on informal consequences, may have limited long-term relevance in contexts where corporate-backed OS software contributions dominate.

Despite this limitation of the long-term relevancy of some of our findings, a strength of our research is that we have based our research on current literature and a qualitative interview study, and have already investigated factors such as legal liability and work status. Both factors might not entirely be characterized as informal consequences and play a minor role today, as only just over 30 % of organizations currently participate in OS software projects ([Wolf, 2022](#)). However, these factors

might become increasingly important in future scenarios if organizations continue to invest in OS software projects. Thus, our research not only reflects the current landscape of OS software projects, where informal consequences dominate, but also anticipates future trends in which formal consequences may take precedence.

Given this evolving dynamic within the landscape of OS software development, further IS research is needed to explore how organizational participation and, with this, formal consequences raised by formal OS governance might replace or complement existing informal consequences. Conducting such future-oriented research is critical to ensuring that OS software projects maintain and ensure high code quality in an era of growing organizational engagement in the OS domain.

6. Conclusion

In the dynamic world of today’s OS software landscape, the integration of OS software projects with high code quality has become indispensable for organizations. However, OS software projects often lack high code quality, posing risks of introducing flawed code and potentially leading to the deployment of compromised algorithmic systems that could pose significant harm to users. Our paper highlights the critical role of developers’ perceived code accountability in shaping their code contributions to OS software projects, particularly how their code accountability perceptions arise in the OS domain, which is usually free of formal consequences, and how it encourages developers to review their code before contributing it. Therefore, our paper helps to understand how informal consequences can compensate for absent or only weakly pronounced formal consequences and motivate developers to perform activities to increase code quality. In the long term, motivating developers to perform activities to increase code quality benefits a majority of users through safer and more reliable algorithmic systems that rely on OS software projects with enhanced code quality.

CRedit authorship contribution statement

Martin Adam: Writing – review & editing, Supervision. **Jan-**

Appendix

Table 8
Interview Guide for the Semi-Structured Online Interviews for Study 1

Number	Question
1	Why do you contribute code to OS software projects?
2	What is your motivation behind your code contributions to OS software projects?
3	Do you perceive yourself accountable for your code when participating in OS software projects? If so, why do you perceive yourself accountable for your code? If not, why do you not perceive yourself accountable for your code?
4	What affects the degree of your perceived accountability for your code? What makes you perceive more or less accountability for your code when participating in OS software projects?
5	How does your perceived accountability for your code affect your behavior and activities when participating in OS software projects?
6	Is there anything else that you would like to talk about?

Table 9
Construct Measures of the Variables

Construct	Items
Perceived Code Accountability (Pre-Vignette) ^a (Hochwarter et al., 2007)	I feel very accountable for the code I contribute to open source projects. I often have to explain my code contributions to open source projects to other developers. In the grand scheme of things, my code contributions to open source projects are very important. To a great extent, the success of open source projects depends on my code contributions.

(continued on next page)

Hendrik Schmidt: Writing – review & editing, Supervision, Conceptualization. **Alexander Benlian:** Writing – review & editing, Supervision, Funding acquisition. **Sebastian Clemens Bartsch:** Writing – original draft, Project administration, Formal analysis, Data curation, Conceptualization.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work, the authors used ChatGPT-3.5 and ChatGPT-4o (chatgpt.com) to improve language and readability. After using this service, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

Declaration of Competing Interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Alexander Benlian reports financial support was provided by German Research Foundation. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The authors gratefully acknowledge the funding support by the German Research Foundation (DFG) as part of the project “Accountable Artificial Intelligence-based Systems” (project number 471168026). In addition, the authors would like to thank the anonymous reviewers and editors for their valuable comments and suggestions for improving this paper.

Table 9 (continued)

Construct	Items
Perceived Code Accountability (Post-Vignette) ^a (Hochwarter et al., 2007)	I would feel very accountable for the code I contribute to cIAIssifier. I believe that I will often have to explain my code contributions to cIAIssifier to other developers. I believe that in the grand scheme of things, my code contributions to cIAIssifier will be very important. I believe that, to a great extent, the success of cIAIssifier depends on my code contributions.
Intention to Review Own Code Contributions ^a (Sykes et al., 2009)	I intend to review my code before contributing it to cIAIssifier. I predict that I will review my code before contributing it to cIAIssifier. I expect that I will review my code before contributing it to cIAIssifier.
Gender ^b	What is your gender?
Age ^c	How old are you? I am __ years old.
Education ^d	What is your highest educational achievement?
Number of Participated OS Software Projects ^c	Approximately how many open source projects have you already contributed code to?
Number of Code Commits to OS Software Projects ^e	Approximately how many code commits have you already contributed to open source projects?
Manipulation Check: Scenario ^f	Within the scenario, I am asked to imagine myself as Alex Peters. Within the scenario, I am asked to imagine being a developer contributing code to an open source project. Within the scenario, cIAIssifier is a commercial software to develop AI systems. You contribute code to an open source project that is integrated into many other projects.
Manipulation Check: Vignette ^f	You are employed by the code provider to contribute code. You have received badges that you can use as certificates for future job applications. You can be held liable by the license.
Scenario-Realism Check ^a (Lapierre, 2007)	The scenario in this study was realistic. The scenario in this study is likely to occur in the real world. The scenario in this study was credible.
Instructional Manipulation Checks ^a (Oppenheimer et al., 2009)	Please select “Strongly agree”.
Nonsensical Questions ^a (Paolacci et al., 2010)	I always sleep twenty-four hours a day. I swim across the Atlantic Ocean every day to get to work.

Note:^a = 7-point Likert-type scale (strongly disagree = 1; strongly agree = 7)

^b = Categorical (Female, Male, Diverse)

^c = Numeric

^d = Ordinal (Finished school with no qualifications, Secondary school-leaving certificate, High school diploma, University/College degree (Bachelor/Master), PhD/Doctoral)

^e = Ordinal (1, 2–10, 11–100, 101–1000, > 1000)

^f = Multiple-selection list box

Table 10

Results of the Indicator Reliability, Internal Consistency, and Convergent Validity

Construct	Indicator Loadings	Cronbach's Alpha	rho _c	AVE
Delta Perceived Code Accountability	0.74–0.86	0.81	0.83	0.64
Intention to Review Own Code Contributions	0.95–0.96	0.95	0.97	0.91

Table 11

Results of the Discriminant Validity (HTMT Values)

Construct	I	II	III	IV	V	VI	VII
I. Delta Perceived Code Accountability							
II. Intention to Review Own Code Contributions	0.64						
III. Age	0.07	0.07					
IV. Gender	0.05	0.17	0.02				
V. Education	0.03	0.10	0.11	0.08			
VI. Number of Participated OS Software Projects	0.19	0.11	0.12	0.04	0.11		
VII. Number of Code Commits to OS Software Projects	0.11	0.13	0.10	0.06	0.38	0.10	

Table 12

Additional Analysis to Test for Discriminant Validity (Fornell-Larcker Criterion)

Construct	I	II	III	IV	V	VI	VII
I. Delta Perceived Code Accountability	0.80						
II. Intention to Review Own Code Contributions	0.56	0.96					
III. Age	−0.06	−0.07	1.00				
IV. Gender	0.02	0.01	−0.17	1.00			
V. Education	−0.01	0.08	0.10	0.11	1.00		
VI. Number of Participated OS Software Projects	0.17	0.11	−0.11	0.12	0.04	1.00	
VII. Number of Code Commits to OS Software Projects	0.10	0.09	0.13	0.10	0.06	0.38	1.00

Data availability

The data that has been used is confidential.

References

- Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G.S., Davis, A., Dean, J., & Devin, M. (2015). Tensorflow: Large-Scale Machine Learning on Heterogeneous Systems.
- Aberdour, M. (2007). Achieving quality in Open-Source software. *IEEE Software*, 24(1), 58–64.
- Ada, M.B., & Majid, M.U. (2022). Developing a System to Increase Motivation and Engagement in Student Code Peer Review. 2022 IEEE International Conference on Teaching, Assessment and Learning for Engineering (TALE), Hong Kong.
- Adam, M. (2022). Accountability-Based User Interface Design Artifacts and Their Implications for User Acceptance of Ai-Enabled Services. European Conference on Information Systems, Timișoara, Romania, 1–16. (<https://aisel.aisnet.org/ecis2022/rp/5/>).
- Ågerfalk, P. J., & Fitzgerald, B. (2008). Outsourcing to an unknown workforce: exploring opensourcing as a global sourcing strategy. *MIS Quarterly*, 32(2), 385–409.
- Ahmed, M. A., & van den Hoven, J. (2010). Agents of Responsibility—Freelance web developers in web applications development. *Information Systems Frontiers*, 12, 415–424.
- Aksulu, A., & Wade, M. R. (2010). A comprehensive review and synthesis of open source research. *Journal of the Association for Information Systems*, 11(11), 576–656.
- Allamanis, M., & Sutton, C. (2013). Mining Source Code Repositories at Massive Scale Using Language Modeling. 10th Working Conference on Mining Software Repositories (MSR), San Francisco, USA, 207–216.
- AlMarzouq, M., Grover, V., & Thatcher, J. B. (2015). Taxing the development structure of open source communities: an information processing view. *Decision Support Systems*, 80, 27–41.
- Anand, A., Krishna, A., Tiwari, R., & Sharma, R. (2018). Comparative Analysis between Proprietary Software Vs. Open-Source Software Vs. Free Software. International Conference on Parallel, Distributed and Grid Computing, Solan, India, 144–147.
- Apache License, Version 2.0, (2004). (<http://www.apache.org/licenses/>).
- Apache Software Foundation. (2017). Cve-2017-5638. CVE. Retrieved 11/26/2023 from (<https://www.cve.org/CVERecord?id=CVE-2017-5638>).
- Bartsch, S.C., Milani, V., Adam, M., & Benlian, A. (2024). Algorithmic Accountability: What Does It Mean for Ai Developers and How Does It Affect Ai Development Projects. Hawaii International Conference on System Sciences, O'ahu, USA, 5826–5835. (<https://hdl.handle.net/10125/107087>).
- Bates, D., Mächler, M., Bolker, B., & Walker, S. (2015). Fitting linear Mixed-Effects models using lme4. *Journal of Statistical Software*, 67(1), 1–48.
- Boehmke, B., Hazen, B., Boone, C. A., & Robinson, J. L. (2020). A data science and open source software approach to analytics for strategic sourcing. *International Journal of Information Management*, 54(102167), 1–11.
- Boulanger, A. (2005). Open-Source versus proprietary software: is one more reliable and secure than the other? *IBM Systems Journal*, 44(2), 239–248.
- Bovens, M. (2007). Analysing and assessing accountability: a conceptual framework. *European Law Journal*, 13, 447–468.
- Brooks, B. (2022, 06/09/2024). Introducing Achievements: Recognizing the Many Stages of a Developer's Coding Journey. (<https://github.blog/2022-06-09-introducing-a-achievements-recognizing-the-many-stages-of-a-developers-coding-journey/>).
- Bubb, A. (2025). Unlocking Ai's Potential: Overcoming Barriers to Adoption. Forbes. Retrieved 02/24/2025 from (<https://www.forbes.com/councils/forbestechcouncil/2025/02/21/unlocking-ais-potential-overcoming-barriers-to-adoption/>).
- Capra, E., Francalanci, C., & Merlo, F. (2008). An empirical study on the relationship between software design quality, development effort and governance in open source projects. *IEEE Transactions on Software Engineering*, 34(6), 765–782.
- Capra, E., Francalanci, C., Merlo, F., & Rossi-Lamastra, C. (2011). Firms' involvement in open source projects: a Trade-Off between software structural quality and popularity. *Journal of Systems and Software*, 84(1), 144–161.
- Cohen, J. (1992). A power primer. *Psychological Bulletin*, 112(1), 155–159.
- Colazo, J., & Fang, Y. (2009). Impact of license choice on open source software development activity. *Journal of the American Society for Information Science and Technology*, 60(5), 997–1011.
- Corbin, J., & Strauss, A. (2014). Basics of Qualitative Research: Techniques and Procedures for Developing Grounded Theory (4 ed.). Sage Publications.
- Dalle, J.-M., David, P.A., Ghosh, R.A., & Wolak, F.A. (2004). Free & Open Source Software Developers and 'the Economy of Regard': Participation and Code-Signing in the Modules of the Linux Kernel. Oxford Workshop on "Libre Source 'convened at the Oxford Internet Institute, Pisa, Italy, 25–26.
- Ellis, P.D. (2010). The Essential Guide to Effect Sizes: Statistical Power, Meta-Analysis, and the Interpretation of Research Results. Cambridge University Press.
- Elo, S., & Kyngäs, H. (2008). The qualitative content analysis process. *Journal of Advanced Nursing*, 62(1), 107–115.
- Faul, F., Erdfelder, E., Buchner, A., & Lang, A.-G. (2009). Statistical power analyses using G*Power 3.1: tests for correlation and regression analyses. *Behavior Research Methods*, 41(4), 1149–1160.
- Filippova, A., Forte, A., Galvez, E., Weiss, R., Dabbish, L., & GitHub. (2017). Open Source Survey. GitHub. Retrieved 06/25/2024 from (<https://opensourcesurvey.org/2017/>).
- Fink, L. (2022). Why and how online experiments can benefit information systems research. *Journal of the Association for Information Systems*, 23(6), 1333–1346.
- Finnigan, K. S., & Gross, B. (2007). Do accountability policy sanctions influence teacher motivation? Lessons from Chicago's Low-Performing schools. *American Educational Research Journal*, 44(3), 594–630.
- Fitzgerald, B. (2006). The transformation of open source software. *MIS Quarterly*, 30(3), 587–598.
- Fuller, C. M., Simmering, M. J., Atinc, G., Atinc, Y., & Babin, B. J. (2016). Common methods variance detection in business research. *Journal of Business Research*, 69(8), 3192–3198.
- Gamblin, T., LeGendre, M., Collette, M.R., Lee, G.L., Moody, A., De Supinski, B.R., & Futral, S. (2015). The Spack Package Manager: Bringing Order to Hpc Software Chaos. Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, New York, NY, USA.
- Germonprez, M., Levy, M., Kendall, J. E., & Kendall, K. E. (2020). Tapestries of innovation: structures of contemporary open source project engagements. *Journal of the Association for Information Systems*, 21(3), 615–663.
- GitHub. (2007). Github. (<https://github.com/>).
- Gitpay. (2024). Gitpay. Retrieved 06/24/2024 from (<https://gitpay.me/>).
- Gobert, M., Nagy, C., Rocha, H., Demeyer, S., & Cleve, A. (2023). Best practices of testing database manipulation code. *Information Systems*, 111, 1–15.
- Goggins, S., Lombard, K., & Germonprez, M. (2021). Open Source Community Health: Analytical Metrics and Their Corresponding Narratives. IEEE/ACM 4th International Workshop on Software Health in Projects, Ecosystems and Communities (SoHeal), Madrid, Spain, 25–33.
- Google. (2008). Android. (<https://developer.android.com/>).
- Google. (2014). Kubernetes. (<https://kubernetes.io/>).
- Gwebu, K. L., Wang, J., & Hu, M. Y. (2020). Information security policy noncompliance: an integrative social influence model. *Information Systems Journal*, 30, 220–269.
- Hair, J.F., Hult, G.T.M., Ringle, C.M., Sarstedt, M., Danks, N.P., & Ray, S. (2021). Partial Least Squares Structural Equation Modeling (PLS-Sem) Using R: A Workbook. Springer International Publishing.
- Hayes, A. F. (2009). Beyond baron and kenny: statistical mediation analysis in the new millennium. *Communication Monographs*, 76(4), 408–420.
- Henseler, J., Ringle, C. M., & Sarstedt, M. (2015). A new criterion for assessing discriminant validity in Variance-Based structural equation modeling. *Journal of the Academy of Marketing Science*, 43, 115–135.
- Hertel, G., Niedner, S., & Herrmann, S. (2003). Motivation of software developers in open source projects: an Internet-Based survey of contributors to the linux kernel. *Research Policy*, 32(7), 1159–1177.
- Ho, S. Y., & Rai, A. (2017). Continued voluntary participation intention in Firm-Participating open source software projects. *Information Systems Research*, 28(3), 603–625.
- Hochwarter, W. A., Ferris, G. R., Gavin, M. B., & Perrew. (2007). Political skill as neutralizer of felt Accountability-Job tension effects on job performance ratings: a longitudinal investigation. *Organizational Behavior and Human Decision Processes*, 102(2), 226–239.
- Horneber, D., & Laumer, S. (2023). Algorithmic accountability. *Business Information Systems Engineering*, 1–8.
- Hornuf, L., & Jeworrek, S. (2023). The effect of community managers on online idea crowdsourcing activities. *Journal of the Association for Information Systems*, 24(1), 222–248.
- Huntr. (2024). Huntr. Retrieved 06/24/2024 from (<https://huntr.com/>).
- James, T. L., Wallace, L., & Deane, J. K. (2019). Using organismic integration theory to explore the associations between Users' exercise motivations and fitness technology feature set use. *MIS Quarterly*, 43(1), 287–312.
- Jasso, G. (2006). Factorial survey methods for studying beliefs and judgments. *Sociological Methods Research*, 34(3), 334–423.
- Keutel, M., Michalik, B., & Richter, J. (2014). Towards mindful case study research in is: a critical analysis of the past ten years. *European Journal of Information Systems*, 23, 256–272.
- Kononenko, O., Baysal, O., & Godfrey, M. W. (2016). Code review quality: how developers see it (Austin, Texas) *Proceedings of the 38th International Conference on Software Engineering*, 1028–1038. <https://doi.org/10.1145/2884781.2884840>.
- Krogh, G., Haefliger, S., Spaeth, S., & Wallin, M. W. (2012). Carrots and rainbows: motivation and social practice in open source software development. *MIS Quarterly*, 649–676.
- Lapierre, L. M. (2007). Supervisor trustworthiness and subordinates' willingness to provide Extra-Role efforts. *Journal of Applied Social Psychology*, 37(2), 272–297.
- MacKinnon, D. (2012). Introduction to Statistical Mediation Analysis. Lawrence Erlbaum Associates.
- Mahmud, H., Islam, A. N., Luo, X. R., & Mikalef, P. (2024). Decoding algorithm appreciation: unveiling the impact of familiarity with algorithms, tasks, and algorithm performance. *Decision Support Systems*, 179, 1–12.
- Markus, M. L. (2007). The governance of Free/Open source software projects: monolithic, multidimensional, or configurational? *Journal of Management Governance*, 11, 151–163.
- Martin, K. (2019). Ethical implications and accountability of algorithms. *Journal of Business Ethics*, 160, 835–850.
- McIntosh, S., Kamei, Y., Adams, B., & Hassan, A. E. (2016). An empirical study of the impact of modern code review practices on software quality. *Empirical Software Engineering*, 21, 2146–2189.
- McLean, R. A., Sanders, W. L., & Stroup, W. W. (1991). A unified approach to mixed linear models. *The American Statistician*, 45(1), 54–64.
- Meta. (2013). React. (<https://react.dev/>).
- 2015 Microsoft Corporation. (2015). Visual Studio Code. (<https://code.visualstudio.com/>).
- Microsoft Corporation. (2018). Presidio. (<https://github.com/microsoft/presidio>).

- Midha, V., & Bhattacharjee, A. (2012). Governance practices and software maintenance: a study of open source projects. *Decision Support Systems*, 54(1), 23–32.
- Mockus, A., Fielding, R. T., & Herbsleb, J. (2000). A case study of open source software development: the apache server. *Proceedings of the 22nd International Conference on Software Engineering*, 263–272 (Ottawa, Canada).
- Möhlmann, M., Zalmanson, L., Henfridsson, O., & Gregory, R. W. (2021). Algorithmic management of work on online labor platforms: when matching meets control. *MIS Quarterly*, 45(4), 1999–2022.
- Nederhof, A. J. (1985). Methods of coping with social desirability bias: a review. *European Journal of Social Psychology*, 15(3), 263–280.
- Newman, L.H. (2021). The Internet Is on Fire. *Wired*. Retrieved 11/26/2023 from (<https://www.wired.com/story/log4j-flaw-hacking-internet/>).
- Novelli, C., Taddeo, M., & Floridi, L. (2023). Accountability in artificial intelligence: what it is and how it works. *AI Society*, 1, 1–12.
- O'Donoghue, D., & Van Der Werff, L. (2022). Empowering leadership: balancing Self-Determination and accountability for motivation. *Personnel Review*, 51(4), 1205–1220.
- O'Mahony, S., & Ferraro, F. (2007). The emergence of governance in an open source community. *Academy of Management Journal*, 50(5), 1079–1106.
- Oppenheimer, D. M., Meyvis, T., & Davidenko, N. (2009). Instructional manipulation checks: detecting satisficing to increase statistical power. *Journal of Experimental Social Psychology*, 45(4), 867–872.
- Palan, S., & Schitter, C. (2018). Prolific.Ac - a subject pool for online experiments. *Journal of Behavioral and Experimental Finance*, 17, 22–27.
- Paolacci, G., Chandler, J., & Ipeirotis, P. G. (2010). Running experiments on Amazon mechanical Turk. *Judgment and Decision Making*, 5(5), 411–419.
- Perens, B. (1999). The Open Source Definition (1 ed.). O'Reilly.
- Perez-Etchegoyen, J. (2023). Application Security Predictions for 2023. *Forbes*. Retrieved 11/26/2023 from (<https://www.forbes.com/sites/forbestechcouncil/2023/03/21/application-security-predictions-for-2023/>).
- Picoto, W. N., Bélanger, F., & Palma-dos-Reis, A. (2014). An organizational perspective on M-Business: usage factors and value determination. *European Journal of Information Systems*, 23(5), 571–592.
- Piquero, N. L., Lyn Exum, M., & Simpson, S. S. (2005). Integrating the Desire-for-Control and rational choice in a corporate crime context. *Justice Quarterly*, 22(2), 252–280.
- Pistilli, G., Muñoz, Ferrandis, C., Jernite, Y., & Mitchell, M. (2023). Stronger together: on the articulation of ethical charters, legal tools, and technical documentation in ml. *Proceedings of the 2023 ACM Conference on Fairness, Accountability, and Transparency*, 343–354.
- Porsche. (2023). Foss Movement: Porsche Anchors Open Source in Software Strategy. Porsche AG. Retrieved 2025-03-27 from (<https://newsroom.porsche.com/en/2023/company/porsche-foss-movements-open-source-software-strategy-33413.html>).
- Proposal for a Regulation of the European Parliament and of the Council Laying Down Harmonised Rules on Artificial Intelligence (Artificial Intelligence Act) and Amending Certain Union Legislative Acts, 1-272 (2024). (<https://data.consilium.europa.eu/doc/document/ST-5662-2024-INIT/en/pdf>).
- Radomir, L., & Moisescu, O. I. (2020). Discriminant validity of the Customer-Based corporate reputation scale: some causes for concern. *Journal of Product Brand Management*, 29(4), 457–469.
- Rashid, M., Clarke, P. M., & O'Connor, R. V. (2019). A systematic examination of knowledge loss in open source software projects. *International Journal of Information Management*, 46, 104–123.
- Recker, J., & La Rosa, M. (2012). Understanding user differences in Open-Source workflow management system usage intentions. *Information Systems*, 37(3), 200–212.
- Red Hat Incorporation. (2018). Cve-2018-10933. CVE. Retrieved 11/26/2023 from (<https://www.cve.org/CVERecord?id=CVE-2018-10933>).
- Richman, W. L., Kiesler, S., Weisband, S., & Drasgow, F. (1999). A Meta-Analytic study of social desirability distortion in Computer-Administered questionnaires, traditional questionnaires, and interviews. *Journal of Applied Psychology*, 84(5), 754–775.
- Rigby, P. C., German, D. M., & Storey, M.-A. (2008). Open source software peer review practices: a case study of the apache server. *30th International Conference on Software Engineering*, 541–550 (Leipzig, Germany).
- Roberts, J. A., Hann, I.-H., & Slaughter, S. A. (2006). Understanding the motivations, participation, and performance of open source software developers: a longitudinal study of the apache projects. *Management Science*, 52(7), 984–999.
- Rolandsson, B., Bergquist, M., & Ljungberg, J. (2011). Open source in the firm: opening up professional practices of software development. *Research Policy*, 40(4), 576–587.
- Rossi, P. H. (1979). Vignette analysis: uncovering the normative structure of complex judgments. In *Qualitative and Quantitative Social Research: Papers in Honor of Paul F. Lazarsfeld* (pp. 176–186). New York: Free Press.
- Rouse, J. (2007). Social practices and normativity. *Philosophy of the Social Sciences*, 37(1), 46–56.
- Schmidt, J.-H., Bartsch, S. C., & Adam, M. (2023). The role of process and outcome accountability claims for shaping ai Developers' perceived accountability. *International Conference on Information Systems*, 1–17. (<https://aisel.aisnet.org/icis2023/isdesign/isdesign/2/>) (Hyderabad, India).
- Schmidt, J.-H., Bartsch, S. C., Adam, M., & Benlian, A. (2025). Elevating Developers' accountability awareness in ai systems development: the role of process and outcome accountability arguments. *Business Information Systems Engineering*, 67, 109–135.
- Schnaack, G., & Termer, F. (2023). Open-Source-Monitor. B. e.V. (<https://www.pwc.de/de/digitale-transformation/bitkom-open-source-monitor-2023.pdf>).
- Sharma, P. N., Daniel, S. L., Chung, T. R., & Grover, V. (2022). A Motivation-Hygiene model of open source software code contribution and growth. *Journal of the Association for Information Systems*, 23(1), 165–195.
- Shrout, P. E., & Bolger, N. (2002). Mediation in experimental and nonexperimental studies: new procedures and recommendations. *Psychological Methods*, 7(4), 422–445.
- Singh, A., Bansal, R., & Jha, N. (2015). Open source software vs proprietary software. *International Journal of Computer Applications*, 114(18), 26–31.
- Spector, P. E., & Brannick, M. T. (2011). Methodological urban legends: the misuse of statistical control variables. *Organizational Research Methods*, 14(2), 287–305.
- Sykes, T. A., Venkatesh, V., & Gosain, S. (2009). Model of acceptance with peer support: a social network perspective to understand employees' system use. *MIS Quarterly*, 33(2), 371–393.
- Synopsys Incorporation. (2023). Open Source Security and Risk Analysis Report. (<https://www.synopsys.com/content/dam/synopsys/sig-assets/reports/rep-ossra-2023.pdf>).
- The MIT License. (1987). (<https://opensource.org/licenses/mit/>).
- Vance, A., Lowry, P. B., & Eggett, D. (2013). Using accountability to reduce access policy violations in information systems. *Journal of Management Information Systems*, 29(4), 263–290.
- Vance, A., Lowry, P. B., & Eggett, D. (2015). Increasing accountability through User-Interface design artifacts. *MIS Quarterly*, 39(2), 345–366.
- Venkatesh, V., Brown, S., & Sullivan, Y. (2024). Conducting Mixed-Methods Research: From Classical Social Sciences to the Age of Big Data and Analytics. Virginia Tech Publishing.
- Wang, C., Li, Y., Chen, L., Huang, W., Zhou, Y., & Xu, B. (2020). Examining the effects of developer familiarity on bug fixing. *Journal of Systems and Software*, 169, Article 110667.
- Wasko, M. M., Faraj, S., & Teigland, R. (2004). Collective action and knowledge contribution in electronic networks of practice. *Journal of the Association for Information Systems*, 5(11), 493–513.
- Whiting, D. (2022). Why Do Enterprises Use and Contribute to Open Source Software. The Linux Foundation. Retrieved 2025-02-22 from (<https://www.linuxfoundation.org/blog/why-do-enterprises-use-and-contribute-to-open-source-software>).
- Wieringa, M. (2020). What to Account for When Accounting for Algorithms: A Systematic Literature Review on Algorithmic Accountability. *Proceedings of the 2020 ACM Conference on Fairness, Accountability, and Transparency*, Barcelona, Spain, 1–18.
- Wolf, A. (2022). Why More Companies Are Investing in Open Source Program Offices. GitHub. Retrieved 02/22/2025 from (<https://github.blog/open-source/maintainers/why-companies-are-starting-ospos/>).
- Wolter, T., Barcomb, A., Riehle, D., & Harutyunyan, N. (2023). Open source license inconsistencies on github. *ACM Transactions on Software Engineering and Methodology*, 32(5), 1–23.
- Wu, C.-G., Gerlach, J. H., & Young, C. E. (2007). An empirical analysis of open source software Developers' motivations and continuance intentions. *Information Management*, 44(3), 253–262.
- Xu, B., Jones, D. R., & Shao, B. (2009). Volunteers' involvement in online community based software development. *Information Management*, 46(3), 151–158.
- Yin, Z., Yuan, D., Zhou, Y., Pasupathy, S., & Bairavasundaram, L. (2011). How Do Fixes Become Bugs? 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering, 26–36.
- Yu, J., Jiang, Z., & Chan, H.C. (2007). Knowledge Contribution in Problem Solving Virtual Communities: The Mediating Role of Individual Motivations. 2007 ACM SIGMIS CPR Conference on Computer Personnel Research: The Global Information Technology Workforce, 144–152.